

DEVOPS PRACTICAL REFERENCE SERIES

PART 01

Linux Administration, Commands & Bash Scripting

Boot process to production troubleshooting: the complete Linux and Bash foundation for a DevOps engineer -- commands, systemd internals, real incident scenarios, and hands-on labs.

Linux

Bash Scripting

systemd

Networking

Security

Troubleshooting

Audience: DevOps engineers with ~2 years of experience

Format: Deep-dive practical reference — concepts, architecture, configuration, commands, troubleshooting, security, automation, and hands-on labs.

How to use: Read sequentially. Each part is self-contained but builds on earlier parts in the series.

Table of Contents

1. Introduction	6
1.1 What This Series Is	6
1.2 Why Linux Is the Foundation of DevOps	6
2. Linux Architecture & the Boot Process	8
2.1 What It Is and Why It Matters	8
2.2 Architecture: The Boot Sequence	8
2.3 Kernel Space vs User Space	9
3. Filesystem Hierarchy & Filesystem Types	11
3.1 The Filesystem Hierarchy Standard (FHS)	11
3.2 Filesystem Types You Will Actually Meet	13
3.3 Inodes: The Detail That Bites Everyone Eventually	13
4. Users, Groups & Permissions	15
4.1 What It Is and Why It Exists	15
4.2 Users and Groups	15
4.3 The Permission Model: rwx for Owner, Group, Others	16
Special bits: SUID, SGID, Sticky	17
4.4 sudo and the Principle of Least Privilege	17
4.5 ACLs: When rwx Isn't Granular Enough	18
5. Process & Service Management	19
5.1 What a Process Is	19
5.2 Inspecting Processes	19
Process states (the STAT column)	19
5.3 Signals	20
5.4 systemd: Deep Dive	21
5.4.1 Units and Targets	21
5.4.2 Writing a Production systemd Unit	22
5.4.3 systemd Timers (the modern cron)	24
5.4.4 cgroups: How systemd (and Docker/Kubernetes) Actually Limit Resources	24
5.5 Priority and Niceness	25
6. Package Management	26
6.1 What It Is and Why It Exists	26
6.2 Debian / Ubuntu: APT and dpkg	26
6.3 RHEL / CentOS / Amazon Linux / Fedora: DNF/YUM and RPM	27
6.4 Repository Management	27
6.5 Cheat Sheet: APT vs DNF/YUM Equivalents	28
7. Disk, Storage & LVM	29
7.1 What It Is and Why It Exists	29

7.2 Partitioning and Formatting	29
7.3 Mounting and /etc/fstab	29
7.4 LVM: Logical Volume Manager	30
8. Networking on Linux	32
8.1 What It Is and Why It Matters	32
8.2 Interface & Address Inspection	32
8.3 Sockets and Listening Ports	32
8.4 DNS Tools	33
8.5 Testing Connectivity and Tracing Paths	33
8.6 Packet Capture: tcpdump	33
8.7 Host-Based Firewalls	34
9. System Monitoring & Performance	35
9.1 What It Is and Why It Exists	35
9.2 CPU and Load	35
9.3 Memory	35
The OOM Killer	36
9.4 The /proc Virtual Filesystem	36
9.5 Disk I/O Performance	37
9.6 Cheat Sheet: “Something’s Wrong” First Five Commands	38
10. Log Management	39
10.1 What It Is and Why It Exists	39
10.2 journalctl: Querying the systemd Journal	39
10.3 rsyslog and /var/log	39
10.4 logrotate: Preventing Disk-Filling Logs	40
11. SSH & Remote Access	41
11.1 What It Is and Why It Exists	41
11.2 Key-Based Authentication	41
11.3 The SSH Client Config	42
11.4 Hardening sshd	42
11.5 Tunneling and File Transfer	43
12. Scheduling: cron, at, and systemd Timers	44
12.1 What It Is and Why It Exists	44
12.2 crontab Syntax	44
12.3 at: One-Off Future Jobs	45
12.4 cron vs systemd Timers vs Kubernetes CronJobs	45
13.1 What It Is and Why It Exists	46
13.2 The Shebang and Execution	46
13.3 Variables, Quoting, and Substitution	46
Quoting rules (the single most common source of Bash bugs)	47
13.4 Conditionals	47

13.5 Loops	48
13.6 Functions	49
13.7 Positional Arguments and getopt	49
13.8 Exit Codes	50
13.9 set -euo pipefail: The Production Standard Header	50
13.10 Traps: Cleanup and Signal Handling	51
14. Text Processing & Real Automation Scripts	52
14.1 What It Is and Why It Exists	52
14.2 grep: Finding Lines	52
14.3 sed: Stream Editing	53
14.4 awk: Field-Based Processing	53
14.5 find and xargs	54
14.6 Other Everyday Tools	54
14.7 Complete Working Script #1: Server Health Check	55
14.8 Complete Working Script #2: Docker Image & Log Cleanup	58
15. Security Hardening Basics	60
15.1 What It Is and Why It Exists	60
15.2 Baseline Hardening Checklist	60
15.3 fail2ban: Automated Brute-Force Protection	60
15.4 SELinux and AppArmor: Mandatory Access Control	61
15.5 Auditing and File Integrity	62
15.6 Security Cheat Sheet: Quick Host Audit	62
16. Troubleshooting Methodology	63
16.1 A Systematic Framework	63
16.2 Common Production Failure Scenarios	63
Scenario: Host load average is very high, but CPU usage looks normal	63
Scenario: “Out of memory” but free -h shows memory available	64
Scenario: Service is “running” (systemctl status = active) but not actually serving traffic	64
Scenario: DNS resolution intermittently fails	64
Scenario: Application crash-loops after a deploy	65
16.3 Systematic Debugging Tool Priority Order	65
17. Hands-On Labs	66
Lab 1 — Boot, systemd, and a Custom Service	66
Lab 2 — Disk Full, Inode Exhaustion, and Recovery	66
Lab 3 — LVM Live Resize	66
Lab 4 — Network Path Debugging	67
Lab 5 — Bash Automation: Log Analyzer	67
Lab 6 — Full Incident Simulation	67
18. Command & Reference Cheat Sheet	68
18.1 Process & Service Management	68

18.2 Disk & Filesystem	68
18.3 Networking	69
18.4 Bash Essentials	69
18.5 Text Processing	69
19. Production Readiness Checklist	70

1. Introduction

1.1 What This Series Is

This is Part 01 of the **DevOps Practical Reference Series** — a set of deep, production-oriented references designed so that, once studied, you should not need to re-learn the same ground from blog posts, random YouTube tutorials, or interview-prep PDFs. Each part in the series assumes roughly two years of hands-on DevOps experience: you have deployed applications, touched a Kubernetes cluster, written a few pipelines, and broken production at least once. The goal here is to convert that scattered experience into structured, durable expertise — deep enough to also let you walk into a 3-year-experience DevOps interview and answer with real, practical confidence, not memorized definitions.

Every major topic in this series follows the same teaching pattern:

1. **What it is and why it exists** — the problem it solves, not just the definition.
2. **Architecture** — how the pieces fit together and talk to each other.
3. **What happens behind the scenes** — the mechanics, not just the interface.
4. **Real production use cases** — where and why you'd actually use this.
5. **Configuration and commands** — with explanations, not just syntax.
6. **Troubleshooting methodology** — how to diagnose, not just a list of fixes.
7. **Security, monitoring, and automation** — treated as first-class citizens, not afterthoughts.
8. **Labs** — so the knowledge becomes muscle memory.

HOW TO USE THIS PART

Part 01 covers Linux administration and Bash scripting — the substrate that every other part in the series (Docker, Kubernetes, Ansible, Terraform, CI/CD) runs on top of. If you are already very strong in Linux internals, skim Sections 2–6 and spend your time on Sections 13–17 (Bash scripting and troubleshooting), which is where most engineers with 2–3 years of experience still have gaps — and where interviewers dig deepest.

1.2 Why Linux Is the Foundation of DevOps

Almost every piece of infrastructure you will touch as a DevOps engineer — cloud VMs, Kubernetes nodes, CI/CD runners, container base images, load balancers, databases — runs Linux. Docker containers are Linux processes isolated with namespaces and cgroups; Kubernetes nodes are Linux hosts running a kubelet; your CI pipeline steps execute inside a Linux shell. If you don't understand Linux at a working level, every layer built on top of it (containers, orchestration, IaC) becomes a black box you can operate but never truly debug.

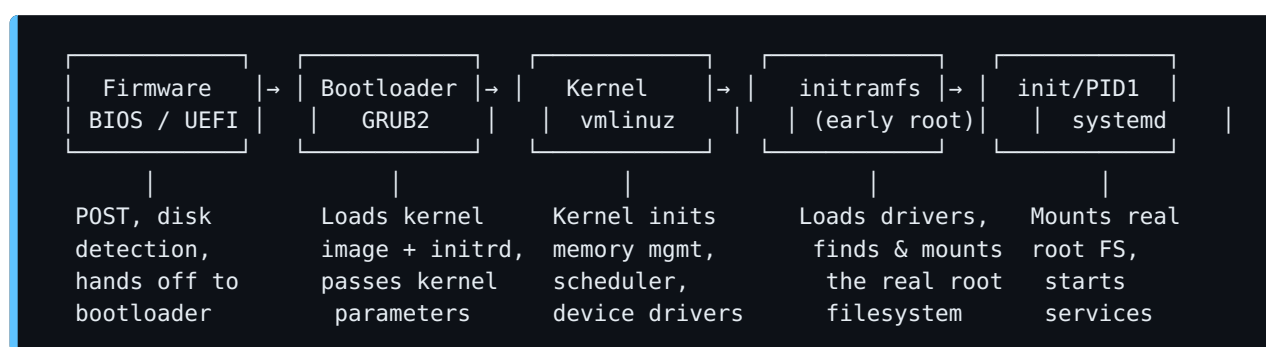
Two years in, most engineers know *commands*. What separates a mid-level engineer from a senior one — and what interviewers are actually probing for — is understanding **why** those commands work the way they do — what the kernel is actually doing when you run `kill -9`, why `df` and `du` can disagree, why a service is “running” but not “ready.” This part builds that mental model.

2. Linux Architecture & the Boot Process

2.1 What It Is and Why It Matters

The boot process is the sequence that takes a powered-off machine to a login prompt. As a DevOps engineer you rarely watch a machine boot from cold, but you constantly deal with its *consequences*: why a cloud instance won't pass health checks after a kernel upgrade, why a container "hangs" at PID 1, why `systemctl` behaves the way it does, why an EC2 instance is stuck in `initializing`. Understanding boot order lets you localize a fault to the right layer instead of guessing.

2.2 Architecture: The Boot Sequence



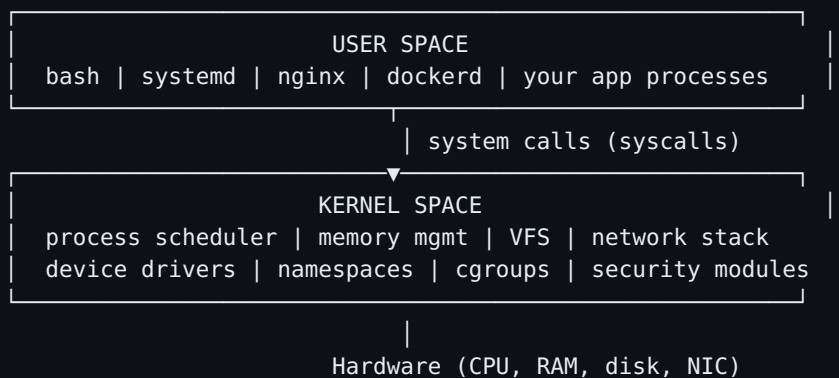
1. **Firmware (BIOS/UEFI):** Runs POST (Power-On Self-Test), initializes hardware, and hands control to the bootloader found on the boot device (MBR for BIOS, EFI System Partition for UEFI).
2. **Bootloader (GRUB2):** Presents a menu (or boots silently on cloud instances), loads the selected Linux **kernel image** (`vmlinuz`) and the **initramfs** into memory, and passes kernel parameters (from `/etc/default/grub`, visible via `cat /proc/cmdline`).
3. **Kernel initialization:** The kernel decompresses itself, initializes core subsystems (memory management, scheduler, interrupt handling), and mounts the **initramfs** as a temporary root filesystem.
4. **initramfs (initial RAM filesystem):** A minimal filesystem in RAM containing just enough drivers/tools (LVM, RAID, disk drivers, cryptsetup) to find and mount the *real* root filesystem — important because the kernel itself may not have built-in drivers for exotic storage (NVMe over Fabric, encrypted LVM root, etc.). Rebuilt with `update-initramfs -u` (Debian/Ubuntu) or `dracut` (RHEL/CentOS) after driver or LVM changes.
5. **Switch root:** The kernel switches from the initramfs to the real root filesystem and executes `/sbin/init`, which on virtually all modern distributions is a symlink to **systemd**.

6. **PID 1 (systemd)**: systemd becomes the very first user-space process (PID 1) and orchestrates everything else — mounting remaining filesystems, starting networking, launching services — by walking a dependency graph of **targets** and **units** (covered in depth in Section 5.4).

PRODUCTION RELEVANCE

Cloud instances (EC2, GCE, Azure VMs) skip firmware/GRUB interaction entirely in normal operation — the hypervisor injects the kernel or uses a pre-baked AMI. But when an instance fails to pass a health check after a reboot, “boot hung” issues almost always trace back to: (a) a broken `/etc/fstab` entry blocking the root mount, (b) a corrupted `initramfs` after a botched kernel/driver update, or (c) a systemd unit with a hard dependency that never becomes “active,” blocking `multi-user.target`. Serial/EC2 console output (`Get System Log`) is the only way to see boot output before networking (and hence SSH) is even up.

2.3 Kernel Space vs User Space



- **User space** processes cannot touch hardware directly; they request kernel services through **system calls** (`read()`, `write()`, `fork()`, `execve()`, `socket()` ...). `strace` shows you exactly which syscalls a process makes — the single most powerful low-level debugging tool available on Linux.
- **Kernel space** is where the scheduler, memory manager, filesystem layer (VFS), network stack, device drivers, and the two subsystems containers are built on — **namespaces** (isolation) and **cgroups** (resource limits) — actually live.

```
# Watch every syscall a command makes – invaluable when a binary "just hangs"
strace -f -tt -o /tmp/trace.out curl -s https://example.com
# -f: follow forked children -tt: microsecond timestamps -o: write to file

# Summarize which syscalls a running process spends time in
strace -c -p <PID>

# Trace only file and network syscalls (less noise)
strace -e trace=open,openat,read,write,connect,accept -p <PID>
```

NOTE

When someone says “it’s hanging,” your first two tools should be `strace -p <pid>` (is it stuck in a syscall — e.g., blocked on a `connect()` that will never return?) and `py-spy` / `jstack` / language-specific profilers if it’s an application-level hang rather than a syscall-level one.

3. Filesystem Hierarchy & Filesystem Types

3.1 The Filesystem Hierarchy Standard (FHS)

Linux organizes everything under a single root () — there are no drive letters. Knowing *why* a directory exists (not just its name) tells you where to look for things and where automation should and shouldn't write.

Path	Purpose	DevOps relevance
<code>/bin</code> , <code>/usr/bin</code>	Essential user binaries	On modern distros <code>/bin</code> is a symlink to <code>/usr/bin</code> (usrmerge)
<code>/sbin</code> , <code>/usr/sbin</code>	System binaries (root-oriented: <code>iptables</code> , <code>fdisk</code>)	Often not in a normal user's <code>\$PATH</code>
<code>/etc</code>	Host-specific configuration	Config management (Ansible) targets this heavily
<code>/var</code>	Variable data: logs, caches, spool, databases	<code>/var/log</code> , <code>/var/lib/docker</code> , <code>/var/lib/mysql</code> — the directory most likely to fill your disk
<code>/var/log</code>	System and application logs	First place to check in almost any incident
<code>/tmp</code>	Temporary files, often cleared on reboot or via <code>systemd-tmpfiles</code>	Never store anything durable here
<code>/opt</code>	Optional/third-party software (self-contained bundles)	Common home for agents: Datadog, Jenkins agents, etc.
<code>/srv</code>	Data served by this host (web/ftp roots)	Some teams put app deployments here instead of <code>/opt</code>
<code>/home</code>	User home directories	SSH keys live in <code>~/.ssh</code> here
<code>/root</code>	root user's home directory	Not under <code>/home</code>
<code>/proc</code>	Virtual filesystem exposing kernel/process state	Not real files — a live window into the kernel (Section 9.4)
<code>/sys</code>	Virtual filesystem exposing kernel/device/driver state	Used to tune kernel parameters, inspect cgroups
<code>/dev</code>	Device files (<code>/dev/sda</code> , <code>/dev/null</code> , <code>/dev/tty</code>)	Managed dynamically by <code>udev</code>
<code>/boot</code>	Kernel image, initramfs, GRUB config	Filling up here blocks kernel upgrades
<code>/mnt</code> , <code>/media</code>	Manual and removable mount points	<code>/mnt</code> for admin-mounted, <code>/media</code> for auto-mounted removable

man hier

the FHS manual page – read this once, seriously

3.2 Filesystem Types You Will Actually Meet

Filesystem	Where you'll see it	Key characteristics
ext4	Default on Debian/Ubuntu for years, most EBS volumes	Journaling, mature, good general performance, supports online resize (grow only)
XFS	Default on RHEL/CentOS/Amazon Linux 2+	Excellent for large files & high parallel I/O, online grow, cannot shrink
Btrfs	openSUSE default, some container use cases	Copy-on-write, native snapshots, subvolumes; more complex, less “boring”
overlay2	Docker/containerd default storage driver	Union filesystem — layers images without duplicating data (Part 06 covers this in depth)
tmpfs	<code>/tmp</code> on many distros, <code>/dev/shm</code> , container <code>emptyDir</code>	Lives in RAM (+ swap); vanishes on unmount/reboot; extremely fast
NFS	Shared storage across hosts, some Kubernetes PVs	Network filesystem; watch for stale file handles and lock issues

```
# What filesystem is mounted where, and how full is it
df -hT
# -h human readable, -T show filesystem type

# Inspect a specific filesystem's superblock (ext4 example)
sudo dumpe2fs -h /dev/xvda1

# Check XFS filesystem info
sudo xfs_info /mnt/data
```

3.3 Inodes: The Detail That Bites Everyone Eventually

Every file is represented by an **inode** — a data structure holding metadata (permissions, owner, timestamps, size, pointers to data blocks) but *not* the filename. Filenames live in **directory entries** that map a name to an inode number. This is why:

- Hard links are just multiple directory entries pointing at the same inode (`ls -li` shows the number).
- You can delete a file that a running process still has open — the inode isn't freed until the last reference (both directory entries *and* open file descriptors) drops to zero. This is precisely why a Java or Nginx process can hold gigabytes of “deleted” log data that `du` reports as gone but `df` reports as still used.

- Filesystems have a **finite number of inodes**, set at format time. You can run out of inodes with plenty of disk space free — a classic production surprise on systems generating huge numbers of tiny files (e.g., a badly configured session-file cache).

```
# Disk space free
df -h /var
# Inode usage free – check this separately!
df -ih /var

# Find which open (deleted) files are silently holding disk space
sudo lsof +L1
# +L1 lists open files with a link count < 1 (i.e., deleted but still held open)
```

COMMON FAILURE: “DF SAYS FULL BUT DU SAYS IT’S FINE”

Classic cause: a process (often a log-writing daemon) has a file open that was deleted by a naive `rm` or a log-rotation script that didn’t signal the process to reopen its log handle. `du` only counts files still linked in the directory tree; `df` reports actual block usage, including blocks held by deleted-but-open files. Fix: find the process with `lsof +L1`, and either restart it (releases the inode) or send it the appropriate reopen signal (e.g., `kill -HUP` for many daemons, or use `logrotate`’s `copytruncate` option going forward).

4. Users, Groups & Permissions

4.1 What It Is and Why It Exists

Linux is a multi-user system at its core, and its permission model is the first and most fundamental security boundary on any host. Every file, process, and socket has an owner and belongs to a group, and every access is checked against a permission model. Getting this wrong is one of the most common causes of both security incidents (over-permissive files/keys) and outages (a service that can't read its own config after a deploy).

4.2 Users and Groups

```
# Key files
cat /etc/passwd      # user accounts: name:x:UID:GID:comment:home:shell
cat /etc/group        # group definitions: name:x:GID:members
cat /etc/shadow       # hashed passwords + aging policy (root-readable only)

# Create a system user for a service (no login shell, no home dir needed)
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appsvc

# Create a normal user with a home directory
sudo useradd -m -s /bin/bash deploy
sudo passwd deploy

# Add a user to a supplementary group (e.g., docker, so they can run docker without
sudo)
sudo usermod -aG docker deploy
# -aG = append to group; forgetting -a and using just -G REPLACES all supplementary
groups – a classic mistake

# See a user's group memberships
id deploy
groups deploy
```

COMMON MISTAKE

`usermod -G docker deploy` (without `-a`) removes the user from every other supplementary group and sets membership to *only* `docker`. This has silently broken sudo access and VPN group membership for engineers more than once. Always use `-aG`.

4.3 The Permission Model: rwx for Owner, Group, Others

```
-rwxr-xr-- 1 deploy devops 8192 Sep 09 10:15 deploy.sh
├─┬─┬─┬─
│ │ │ │
│ │ │ └─ others: r-- (read only)
│ │ └─── group:  r-x (read + execute)
│ └───── owner:  rwx (read + write + execute)
└──────── file type: - regular file, d directory, l symlink, etc.
```

Symbol	Numeric	Meaning on a file	Meaning on a directory
r	4	Read file contents	List directory contents (<code>ls</code>)
w	2	Modify file contents	Create/delete/rename entries inside
x	1	Execute the file	Enter the directory (<code>cd</code>) / traverse it

```
# Numeric (absolute) permission setting
chmod 750 deploy.sh           # rwxr-x---
chmod -R 640 /etc/myapp/      # recursive; careful – directories need execute (x) to be
enterable!
chmod ug+x run.sh             # symbolic: add execute for user+group
chmod o-rwx secrets.env       # remove all access for "others"

# Ownership
chown deploy:devops app.jar
chown -R deploy: /opt/myapp/   # keep existing group, change owner only
chgrp devops shared_dir/

# umask – the default permission MASK applied to newly created files/dirs
umask           # e.g., 0022 → new files 644 (666-022), new dirs 755 (777-022)
umask 027       # tighter default for a service account: files 640, dirs 750
```

Special bits: SUID, SGID, Sticky

Bit	On files	On directories	Numeric prefix
SUID (4xxx)	Runs the file with the owner's privileges, not the caller's (e.g., <code>/usr/bin/passwd</code> runs as root so any user can update <code>/etc/shadow</code>)	Ignored	<code>chmod 4755 file</code>
SGID (2xxx)	Runs with the group's privileges	New files inherit the directory's group (very useful for shared team directories)	<code>chmod 2775 dir</code>
Sticky (1xxx)	Ignored	Only the file's owner (or root) can delete/rename files inside, even if others have write access (this is why <code>/tmp</code> is <code>1777</code>)	<code>chmod 1777 dir</code>

```
ls -ld /tmp
# drwxrwxrwt ... /tmp    ← the trailing "t" is the sticky bit

find / -xdev -perm -4000 -type f 2>/dev/null    # audit: find all SUID binaries on a host
```

SECURITY NOTE

SUID binaries are a classic privilege-escalation vector. Any custom SUID root binary on a production host should be treated as a security review item. Regularly audit with the `find` command above and compare against a known-good baseline.

4.4 sudo and the Principle of Least Privilege

```
sudo visudo
# ALWAYS edit sudoers via visudo – it validates syntax before saving

# /etc/sudoers or /etc/sudoers.d/deploy – grant narrow, specific access, not blanket root
deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart myapp, /usr/bin/systemctl status myapp

# Prefer drop-in files over editing /etc/sudoers directly
sudo visudo -f /etc/sudoers.d/deploy
```

PRODUCTION PRACTICE

Never hand out `NOPASSWD: ALL` to a service or deploy account. Scope sudo rules to the exact binaries and even exact arguments a CI/CD deploy user needs (e.g., restarting one named service), and put those rules in version-controlled Ansible/Terraform, not manual edits, so drift is visible in code review.

4.5 ACLs: When rwx Isn't Granular Enough

Standard Unix permissions only support one owner and one group. **Access Control Lists (ACLs)** let you grant permissions to additional specific users or groups on a single file/directory.

```
# Grant read+write to a specific user, without changing the file's group
setfacl -m u:jenkins:rw /opt/deployments/artifact.jar

# Grant a group access
setfacl -m g:qa-team:rx /opt/releases/

# View ACLs
getfacl /opt/deployments/artifact.jar

# Remove all ACL entries
setfacl -b /opt/deployments/artifact.jar
```

5. Process & Service Management

5.1 What a Process Is

A process is a running instance of a program: it has a **PID**, memory space, open file descriptors, a parent (**PPID**), a set of resource limits (**ulimit**), and a scheduling priority. Understanding process state and signaling is core to diagnosing “my service is stuck / using too much CPU / won’t die.”

5.2 Inspecting Processes

```
ps aux                # BSD-style: all processes, all users, full detail
ps -ef               # SysV-style equivalent
ps -eo pid,ppid,%cpu,%mem,stat,etime,cmd --sort=-%cpu | head # top CPU consumers,
custom columns

top                  # live view; press '1' for per-core, 'M' sort by
memory, 'k' to kill
htop                 # nicer interactive version (tree view, mouse, easy
kill) – install if missing

pstree -p           # visualize the process tree with PIDs
```

Process states (the **STAT** column)

Code	Meaning
R	Running or runnable (on the CPU run queue)
S	Interruptible sleep (waiting for an event — most idle processes)
D	Uninterruptible sleep — usually waiting on disk/NFS I/O; cannot be killed with normal signals
T	Stopped (e.g., via Ctrl+Z or SIGSTOP)
Z	Zombie — process has exited but its parent hasn’t reaped its exit status yet

COMMON FAILURE: PROCESSES STUCK IN D STATE

A process in **D** state is blocked on a kernel-level I/O operation (disk, NFS) and cannot be killed — not even with **kill -9** — until the I/O completes or times out. If you see many processes piling up in **D**, the actual problem is almost always the underlying storage (a hung NFS mount, a failing disk, an overloaded EBS volume) — kill signals will not fix it; you must fix or fail over the storage.

ZOMBIES VS ORPHANS

A **zombie** has exited but its exit code hasn't been collected by its parent (`wait()` not called) — it holds almost no resources (just a PID table slot) and cannot be killed directly; fix the parent, or if the parent itself dies, `init` / `systemd` (PID 1) adopts and reaps it. An **orphan** is simply a process whose parent died while it kept running — it gets re-parented to PID 1, which is normal and harmless. Large numbers of zombies indicate a buggy parent process that never calls `wait()` — very common with badly written shell scripts or supervisors that fork children without reaping them.

5.3 Signals

Signals are the mechanism the kernel and other processes use to communicate events to a process — not just “stop.”

Signal	Number	Default action	Typical use
<code>SIGHUP</code>	1	Terminate (many daemons: reload config instead)	<code>kill -HUP <pid></code> to make <code>nginx/rsyslog</code> re-read config
<code>SIGINT</code>	2	Terminate	What <code>Ctrl+C</code> sends
<code>SIGKILL</code>	9	Terminate immediately, uncatchable	Last resort — the process cannot clean up, no graceful shutdown
<code>SIGTERM</code>	15	Terminate	The polite/default request — <code>kill <pid></code> and <code>docker stop</code> send this first; well-behaved apps flush buffers, close connections gracefully
<code>SIGSTOP</code>	19	Suspend, uncatchable	Pause a process
<code>SIGCONT</code>	18	Resume	Resume after <code>SIGSTOP</code>
<code>SIGUSR1</code> / <code>SIGUSR2</code>	10/12	Terminate (custom apps redefine)	App-defined actions (e.g., trigger a log rotation, dump debug state)

```
kill -15 <pid>          # SIGTERM – always try this first
kill -9 <pid>           # SIGKILL – only when SIGTERM is ignored/hung; skips cleanup
                        (open files, temp data)
kill -l                # list all signal names/numbers
pkill -f "python worker.py" # kill by matching the command line
killall nginx          # kill all processes by exact name

# Send SIGTERM, wait, escalate to SIGKILL only if still alive – the pattern most
# orchestrators use
kill -15 <pid>; sleep 10; kill -9 <pid> 2>/dev/null && kill -9 <pid>
```

WHY THIS MATTERS FOR KUBERNETES/DOCKER

When Kubernetes deletes a Pod, it sends **SIGTERM**, waits up to **terminationGracePeriodSeconds** (default 30s), then sends **SIGKILL**. If your application doesn't handle **SIGTERM** (e.g., it's PID 1 in a container using the default shell-form **CMD**, which doesn't forward signals), it gets hard-killed every time, dropping in-flight requests. This is one of the most common causes of failed rolling deployments and is covered in depth in Part 06 (Docker) and Part 09 (K8s Workloads).

5.4 systemd: Deep Dive

systemd is PID 1 on virtually every modern distribution (Ubuntu, Debian, RHEL/CentOS 7+, Amazon Linux 2+). It replaced the old SysV init scripts with a **declarative, parallelized, dependency-driven** service manager.

5.4.1 Units and Targets

A **unit** is anything systemd manages: a service, a mount, a socket, a timer, a device, or a target (a synchronization point/group of units — the rough equivalent of an old-style runlevel).

Unit type	Extension	Purpose
Service	.service	A managed process/daemon
Socket	.socket	Socket-activated services (start on first connection)
Mount / Automount	.mount / .automount	Filesystem mount points
Timer	.timer	Scheduled activation of another unit (cron replacement)
Target	.target	Group/synchronization point, e.g. multi-user.target , graphical.target
Path	.path	Activate a unit when a file/directory changes

```

systemctl list-units --type=service --state=running      # what's actually running
systemctl list-unit-files --state=enabled                # what's set to start on boot
systemctl get-default                                    # current default target
(e.g., multi-user.target)
systemctl list-dependencies multi-user.target            # what a target pulls in,
as a tree

```

5.4.2 Writing a Production systemd Unit

```

# /etc/systemd/system/myapp.service
[Unit]
Description=My Application API server
Documentation=https://internal-wiki/myapp
After=network-online.target      # ordering: start after networking is up
Wants=network-online.target      # soft dependency: try to have it, don't hard-
fail if missing
StartLimitIntervalSec=300        # rate-limit restart attempts (see
StartLimitBurst)
StartLimitBurst=5

[Service]
Type=notify                      # notify systemd via sd_notify() when actually
ready (see Type= table below)
User=appsvc
Group=appsvc
WorkingDirectory=/opt/myapp
EnvironmentFile=/etc/myapp/myapp.env  # KEY=VALUE pairs loaded as env vars
ExecStartPre=/opt/myapp/bin/preflight-check.sh
ExecStart=/opt/myapp/bin/myapp --config /etc/myapp/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure               # restart policy (see table below)
RestartSec=5
TimeoutStopSec=30                # grace period before SIGKILL on stop
LimitNOFILE=65536                # raise the open-file-descriptor ulimit for
this service

# Security hardening (sandbox the service – see Section 15)
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/myapp
PrivateTmp=true

[Install]
WantedBy=multi-user.target

```

Type=	Meaning
<code>simple</code> (default)	systemd considers the service started as soon as <code>ExecStart</code> is forked — doesn't know if the app is actually <i>ready</i>
<code>forking</code>	For classic daemons that fork and exit the parent (e.g., legacy <code>httpd</code>)
<code>oneshot</code>	Runs to completion, then exits (good for setup scripts, migrations)
<code>notify</code>	Service calls <code>sd_notify(READY=1)</code> when truly ready — the most accurate for health-aware startup ordering

Restart=	Meaning
<code>no</code> (default)	Never auto-restart
<code>on-failure</code>	Restart only on non-zero exit / signal / timeout — the usual production choice
<code>always</code>	Restart no matter how it exited, including clean exit
<code>on-abnormal</code>	Restart on signal/timeout/watchdog, not on clean or unclear exit codes

```

sudo systemctl daemon-reload      # re-read unit files after any edit – easy to
forget!
sudo systemctl enable --now myapp.service  # enable at boot AND start now
sudo systemctl status myapp.service      # state, recent logs, cgroup, memory
sudo systemctl restart myapp.service
sudo systemctl edit myapp.service        # create an override drop-in without
touching the original file
journalctl -u myapp.service -f           # follow this unit's logs live
journalctl -u myapp.service --since "10 min ago" -p err # only errors, last 10
minutes

```

COMMON MISTAKE

Editing a unit file directly under `/etc/systemd/system/` and forgetting `systemctl daemon-reload` — systemd keeps running with the old, cached unit definition, and your change silently doesn't apply until the next reload or reboot. Prefer `systemctl edit`, which creates a drop-in override and reminds you.

5.4.3 systemd Timers (the modern cron)

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup.service daily at 02:00

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true           # if the machine was off at 02:00, run it as soon as it's back
up
RandomizedDelaySec=300    # spread load if this unit is templated across many hosts

[Install]
WantedBy=timers.target
```

```
systemctl enable --now backup.timer
systemctl list-timers           # see all timers and their next scheduled run
```

Timers are generally preferable to cron for anything tied to a systemd-managed service: you get dependency ordering, resource limiting (via the same `[Service]` directives as any unit, in a paired `.service` file), structured logging in the journal, and `systemctl list-timers` visibility — versus cron’s silent failures unless you build your own alerting.

5.4.4 cgroups: How systemd (and Docker/Kubernetes) Actually Limit Resources

Control groups (cgroups) are the kernel mechanism that limits, accounts for, and isolates resource usage (CPU, memory, I/O, PIDs) for a group of processes. systemd creates a cgroup **per unit** automatically; Docker and Kubernetes use the exact same mechanism per container/pod.

```
# See the cgroup a systemd service lives in and its live resource accounting
systemctl status myapp.service           # shows CGroup: path near the bottom
cat /sys/fs/cgroup/system.slice/myapp.service/memory.current
# cgroup v2 path (live memory usage, bytes)
cat /sys/fs/cgroup/system.slice/myapp.service/memory.max           # configured hard
limit

# Set resource limits directly on a systemd unit (no need for a separate cgroup tool)
systemctl set-property myapp.service MemoryMax=512M CPUQuota=50%
```

NOTE

This is the exact mechanism behind Kubernetes `resources.limits` (Part 13) and Docker’s `--memory` / `--cpus` flags (Part 06) — they all ultimately write to the same cgroup interface. When a container gets OOM-killed, it is the kernel’s cgroup OOM killer enforcing `memory.max`, not some Docker/Kubernetes-specific mechanism.

5.5 Priority and Niceness

```
nice -n 10 ./batch_job.sh          # start a new process with lower CPU priority
                                   (higher niceness = lower priority; range -20 to 19)
renice -n 5 -p 1234                # change priority of an already-running process
ionice -c2 -n7 -p 1234             # I/O scheduling class/priority (class 2 = best-
                                   effort, 0-7, 7 = lowest)
```

Use `nice` / `ionice` for background batch jobs (backups, log compaction, report generation) that should never starve interactive/production workloads sharing the same host.

6. Package Management

6.1 What It Is and Why It Exists

Package managers install, upgrade, remove, and track software along with its dependency graph, checksums, and metadata — turning “download a tarball and hope” into a reproducible, auditable operation. Understanding the two major families (Debian/APT and RedHat/YUM-DNF) and their lower-level tools (`dpkg` , `rpm`) is essential for building base images, debugging failed installs in CI, and pinning versions for reproducibility.

6.2 Debian / Ubuntu: APT and dpkg

```
sudo apt update                # refresh the package index from
configured repos (/etc/apt/sources.list*)
sudo apt upgrade               # upgrade installed packages to latest
available
sudo apt install nginx=1.24.0-2ubuntu1  # install a SPECIFIC pinned version –
critical for reproducible builds
sudo apt remove nginx          # remove package, keep config files
sudo apt purge nginx           # remove package AND its config files
sudo apt autoremove            # remove orphaned dependencies no
longer needed
apt-cache policy nginx
# see installed vs candidate versions and which repo they come from
apt-cache search "redis"      # search package names/descriptions
apt list --installed | grep nginx  # check if/what's installed
dpkg -L nginx                  # list every file a package
installed
dpkg -S /usr/sbin/nginx        # reverse lookup: which package owns this file
dpkg -i local-package.deb      # install a local .deb file
directly
```

```
# Pin a package to prevent accidental upgrades (apt-mark, or /etc/apt/preferences.d/)
sudo apt-mark hold nginx
sudo apt-mark unhold nginx
```

DOCKERFILE BEST PRACTICE

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl=7.81.0-1ubuntu1.15 \
    ca-certificates && \
    rm -rf /var/lib/apt/lists/*
```

Combine `update` and `install` in a single `RUN` layer (so a stale cached index is never used for a later install), pin versions for reproducibility, use `--no-install-recommends` to avoid bloating the image, and clean the apt cache in the same layer to keep image size down.

6.3 RHEL / CentOS / Amazon Linux / Fedora: DNF/YUM and RPM

```
sudo dnf check-update                # like apt update – list available
updates
sudo dnf install httpd-2.4.57        # install a specific version
sudo dnf remove httpd
sudo dnf list installed | grep httpd
sudo dnf history                     # transaction history – supports
rollback!                           # roll back a bad update
sudo dnf history undo <transaction-id> # query installed packages
rpm -qa | grep httpd
(work even if dnf/yum is broken)
rpm -ql httpd                       # list files owned by a
package
rpm -qf /usr/sbin/httpd             # reverse lookup: which
package owns this file
rpm -ivh package.rpm                # install a local RPM
directly
sudo dnf versionlock add httpd
# pin a package version (needs the versionlock plugin)
```

6.4 Repository Management

```
# Debian/Ubuntu – add a repo (modern, keyring-based, avoids apt-key deprecation issues)
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/
apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update

# RHEL/CentOS – add a repo
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-
ce.repo
```

SECURITY CONSIDERATION

Never disable GPG signature checking (`-nogpgcheck` , `[trusted=yes]`) to “get past” a repo error in production — you’re removing the mechanism that verifies packages haven’t been tampered with. Fix the actual key/repo issue instead. This matters even more when you consider supply-chain security (Part 26).

6.5 Cheat Sheet: APT vs DNF/YUM Equivalents

Task	APT (Debian/Ubuntu)	DNF/YUM (RHEL/CentOS/ Amazon Linux)
Refresh index	<code>apt update</code>	<code>dnf check-update</code>
Install	<code>apt install pkg</code>	<code>dnf install pkg</code>
Remove	<code>apt remove pkg</code>	<code>dnf remove pkg</code>
Remove + config	<code>apt purge pkg</code>	<code>dnf remove pkg</code> (RPM has no separate purge)
Search	<code>apt search term</code>	<code>dnf search term</code>
List installed	<code>dpkg -l</code> / <code>apt list --installed</code>	<code>rpm -qa</code>
Which pkg owns file	<code>dpkg -S /path</code>	<code>rpm -qf /path</code>
List files in pkg	<code>dpkg -L pkg</code>	<code>rpm -ql pkg</code>
Clean cache	<code>apt clean</code>	<code>dnf clean all</code>
Local file install	<code>dpkg -i file.deb</code>	<code>rpm -ivh file.rpm</code>

7. Disk, Storage & LVM

7.1 What It Is and Why It Exists

Storage management covers partitioning raw block devices, formatting them with a filesystem, mounting them into the directory tree, and — for anything beyond the simplest case — using **LVM (Logical Volume Manager)** to add a flexible layer between physical disks and filesystems so you can resize, snapshot, and span volumes without downtime.

7.2 Partitioning and Formatting

```
lsblk                                # tree view of block devices, partitions, mount
points – start here
lsblk -f                             # same, plus filesystem type and UUID

sudo fdisk /dev/xvdb                 # classic MBR/GPT partitioning tool (interactive:
n=new, p=print, w=write)
sudo parted /dev/xvdb --script mklabel gpt mkpart primary ext4 0% 100% # scriptable,
GPT-native alternative

sudo mkfs.ext4 /dev/xvdb1            # format a new partition as ext4
sudo mkfs.xfs /dev/xvdb1            # format as XFS

sudo blkid /dev/xvdb1               # show the filesystem's UUID and type – use
UUID, not device name, in fstab
```

7.3 Mounting and /etc/fstab

```
sudo mkdir -p /data
sudo mount /dev/xvdb1 /data          # mount NOW (does not survive reboot on its
own)
mount | grep /data                  # verify

sudo umount /data                   # unmount (fails if something has an open
file/cwd there – see lsof)
```

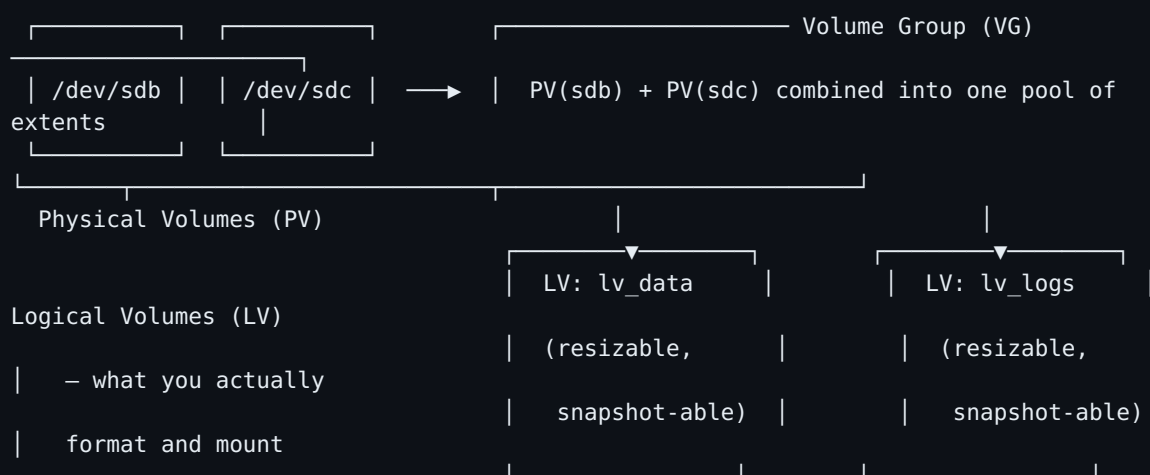
```
# /etc/fstab – mounted automatically at every boot
# <device>          <mountpoint> <fstype> <options>
<dump> <pass>
UUID=1b4e28ba-2fa1-11d2-883f-b9a761bf3fa2 /data          ext4          defaults,noatime
0          2
```

PRODUCTION LANDMINE

A bad entry in `/etc/fstab` (wrong UUID, unreachable NFS server without the `nofail` option) can hang the boot process entirely on next reboot, because by default the kernel will not proceed to a multi-user target until all non-`nofail` filesystems mount successfully. Always: (1) test with `sudo mount -a` before rebooting after any `fstab` edit, and (2) add `nofail` (and often `_netdev` for network filesystems) to any non-critical or network-backed mount.

```
sudo mount -a          # mount everything in fstab NOT already mounted – the
                        # standard fstab validation step
findmnt /data          # confirm what's actually mounted and with which
                        # options, tree-formatted
```

7.4 LVM: Logical Volume Manager



LVM adds three layers between raw disks and filesystems: **Physical Volumes** (raw disks/partitions), **Volume Groups** (pools combining PVs), and **Logical Volumes** (resizable “virtual partitions” carved from a VG). This is what lets you grow a filesystem online by adding a new disk to the VG and extending the LV — no downtime, no repartitioning.

```

# Build the stack from scratch
sudo pvcreate /dev/xvdc /dev/xvdd      # mark raw disks as physical volumes
sudo vgcreate vg_data /dev/xvdc /dev/xvdd  # combine PVs into a volume group
sudo lvcreate -L 50G -n lv_app vg_data    # carve out a 50GB logical volume
sudo mkfs.xfs /dev/vg_data/lv_app        # format it like any block device
sudo mount /dev/vg_data/lv_app /opt/app

# Grow it online, without downtime, when you run low on space
sudo lvextend -L +20G /dev/vg_data/lv_app  # grow the logical volume by 20GB
sudo xfs_growfs /opt/app                  # XFS: grow the filesystem to fill
the LV (XFS can only grow, never shrink)
sudo resize2fs /dev/vg_data/lv_app        # ext4 equivalent

# Inspect the stack
sudo pvs  # physical volumes
sudo vgs  # volume groups
sudo lvs  # logical volumes

```

PRODUCTION SCENARIO

“The database disk is at 95% and we can’t take downtime.” With LVM: attach a new EBS/cloud disk → **pvcreate** it → **vgextend** the existing VG → **lvextend** the LV → **xfs_growfs** / **resize2fs** the mounted filesystem live. Total downtime: zero. Without LVM on a plain partition, you’re looking at a much more painful migration. This is precisely why almost every cloud “data” volume layout uses LVM by default.

8. Networking on Linux

8.1 What It Is and Why It Matters

Every service you run, every container, every Kubernetes Pod ultimately depends on the host's network stack. Being fluent with `ip`, `ss`, `curl`, `dig`, and `tcpdump` is what separates “the app can't connect, I'll ask the network team” from independently proving *where* in the path a connection is failing within minutes.

8.2 Interface & Address Inspection

```
ip addr show                # (ip a) list interfaces and their IPs – the modern
                             replacement for ifconfig
ip link show                # interface state (UP/DOWN), MAC address, MTU
ip route show
# (ip r) the routing table – what gateway traffic to a given destination uses
ip route get 8.8.8.8        # which interface/gateway would be used to
                             reach this specific IP

# Temporarily bring an interface up/down, or add an IP (not persistent – use netplan/
NetworkManager/nmcli for that)
sudo ip link set eth0 up
sudo ip addr add 10.0.0.5/24 dev eth0
```

8.3 Sockets and Listening Ports

```
ss -tulpn
# -t tcp -u udp -l listening only -p show owning process -n numeric ports (skip
slow DNS lookups)

ss -tan state established   # all established TCP connections
ss -s                      # summary statistics (total sockets, TCP
states breakdown)

# netstat is the legacy equivalent (often not installed by default anymore, but still
widely referenced)
netstat -tulpn
```

NOTE

“Is anything listening on port 8080?” is one of the single most common first debugging steps for a “connection refused” error. `ss -tulpn | grep 8080` answers it in one line and additionally tells you the exact PID/process — `curl` ing the port from the box itself is the very next step.

8.4 DNS Tools

```
dig example.com                # full DNS query + answer, authority, and timing
detail
dig +short example.com         # just the resolved IP(s) – good for scripting
dig example.com MX             # query a specific record type
dig @8.8.8.8 example.com       # query a specific DNS server directly,
bypassing local resolver – isolates "is it MY resolver or the record itself?"
dig -x 8.8.8.8                 # reverse DNS lookup

nslookup example.com           # older tool, still common in minimal/
Windows-influenced environments
host example.com               # quick, terse lookup

cat /etc/resolv.conf           # which DNS servers this host is
configured to use
systemd-resolve --status
# on systemd-resolved systems, the effective resolver config per interface
```

8.5 Testing Connectivity and Tracing Paths

```
ping -c 4 example.com          # basic reachability + latency (ICMP – often
blocked by firewalls, so a failure here isn't conclusive)
traceroute example.com         # hop-by-hop path – where does it stop
responding?
mtr example.com                # traceroute + ping combined, continuously
updating – the best of both, use this over traceroute when available

curl -v https://api.example.com/health # verbose: shows DNS, TCP connect, TLS
handshake, request, response – the single best "is the whole chain OK" tool
curl -o /dev/null -s -w "dns:%{time_namelookup} connect:%{time_connect} ttfb:%
{time_starttransfer} total:%{time_total}\n" https://api.example.com
# breaks down EXACTLY where time is spent: DNS resolution vs TCP connect vs TLS vs
waiting for the server vs downloading

nc -zv example.com 443         # quick TCP port-open check ("zero-I/O" scan
mode) – is the port even reachable, ignoring HTTP entirely
telnet example.com 443        # older alternative for the same raw-TCP check
```

8.6 Packet Capture: tcpdump

```
sudo tcpdump -i eth0 -n port 443
# capture on eth0, no DNS resolution (-n = faster, cleaner), filter to port 443
sudo tcpdump -i any host 10.0.1.5 and port 5432 # filter to a specific host+port
(e.g., debugging a Postgres connection)
sudo tcpdump -i eth0 -w capture.pcap           # write to a file for later
analysis in Wireshark
sudo tcpdump -i eth0 -A port 80                # -A: print packet contents
in ASCII (readable HTTP headers/body)
```

PRODUCTION SCENARIO: “THE APP CAN’T REACH THE DATABASE”

A systematic path: (1) `ping / mtr` the DB host — is the network path up at all? (2) `nc -zv db-host 5432` — is the TCP port reachable (rules out app-layer issues, isolates to network/firewall/security-group)? (3) if the port is closed/unreachable, check security groups/firewall rules, not the app config. (4) if the port IS open but the app still fails, the issue is authentication/app-layer — `tcpdump` on the DB host itself while retrying from the app will show whether the SYN even arrives. This sequence — L3 (ping) → L4 (port) → L7 (app) — is the general model for isolating any “can’t connect” issue at any layer of the stack, including inside Kubernetes (Part 10 applies the identical model to Service/Pod networking).

8.7 Host-Based Firewalls

```
# iptables (legacy but still the underlying engine on many systems, and what Docker
manipulates directly)
sudo iptables -L -n -v --line-numbers          # list all rules, numeric, with hit
counters
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -j DROP                 # default-deny at the end of the
chain

# nftables – the modern replacement for iptables on current distros
sudo nft list ruleset

# ufw – a friendly wrapper over iptables (Debian/Ubuntu)
sudo ufw allow 22/tcp
sudo ufw allow from 10.0.0.0/8 to any port 5432
sudo ufw enable
sudo ufw status verbose

# firewalld – the RHEL/CentOS/Amazon Linux equivalent, zone-based
sudo firewall-cmd --add-service=https --permanent
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

COMMON MISTAKE

Enabling `ufw / firewalld` over an SSH session without first confirming a rule that allows your current SSH port — you will lock yourself out immediately. Always add and verify the SSH allow rule *before* enabling the firewall, and test the new rule from a second, still-connected session before closing the first.

9. System Monitoring & Performance

9.1 What It Is and Why It Exists

Before any external monitoring stack (Part 22: Prometheus/Grafana) even connects, the raw ability to read a Linux host's live vital signs from the command line — CPU, memory, disk I/O, and the kernel's own live state via `/proc` — is what lets you triage an incident in the first sixty seconds, and it's what those monitoring tools are built on top of internally.

9.2 CPU and Load

```
uptime
# 14:32:01 up 21 days,  3:12,  2 users,  load average: 0.15, 0.22, 0.19
# load average = the 1/5/15-minute average number of processes RUNNABLE or in
UNINTERRUPTIBLE (D-state) sleep

nproc                # number of CPU cores visible to this process/container
mpstat -P ALL 1      # per-core CPU utilization, refreshed every second
(needs sysstat package)
vmstat 1 5           # 5 samples, 1 second apart – r (runnable), b
(blocked/D-state), CPU us/sy/id/wa columns
```

READING LOAD AVERAGE CORRECTLY

Load average is NOT the same as “CPU percent busy.” A load of 4.0 on an 8-core box means the system is comfortably under capacity; a load of 4.0 on a 2-core box means the system is badly overloaded, with processes queued waiting for CPU. Always divide load by `nproc` to get a normalized utilization figure. Also remember load average counts processes waiting on disk I/O (D-state) too — a load spike with *low* CPU usage almost always points to a storage bottleneck, not a compute one.

9.3 Memory

```
free -h              # total/used/free/shared/buff-cache/available, human-
readable
# "available" (not "free") is the number that actually matters – it accounts for
reclaimable cache

cat /proc/meminfo     # the raw, complete source data free/top read from

# Top memory consumers
ps -eo pid,comm,%mem,rss --sort=-rss | head -10
```

NOTE: “FREE” BEING LOW IS USUALLY FINE

Linux aggressively uses spare RAM for disk/page cache to speed up I/O — that memory is immediately reclaimable if an application needs it. A host showing 200MB “free” but 6GB “available” is healthy, not starved. The classic beginner mistake is alerting on low “free” memory instead of low “available” memory.

The OOM Killer

When the kernel truly cannot satisfy a memory allocation (available memory + swap exhausted), it invokes the **OOM (Out-Of-Memory) killer**, which selects a process to kill based on an `oom_score` (roughly: memory footprint, weighted by priority) to free memory and keep the system alive.

```
dmesg -T | grep -i "out of memory"      # confirms an OOM kill happened and shows the victim
grep -i oom /var/log/syslog              # (or journalctl -k) same evidence, syslog-side
cat /proc/<pid>/oom_score
# this process's current OOM-kill likelihood (higher = more likely to be killed)
echo -1000 > /proc/<pid>/oom_score_adj   # make a critical process effectively immune to the OOM killer (use sparingly)
```

PRODUCTION RELEVANCE

This is exactly what happens inside a Kubernetes Pod that hits its memory `limit` — the cgroup OOM killer fires and the container is killed with exit code 137 (128 + SIGKILL’s signal number 9). “OOMKilled” in `kubectl describe pod` is this exact mechanism, just scoped to a cgroup instead of the whole host. Part 13 covers sizing requests/limits to avoid this in the first place.

9.4 The /proc Virtual Filesystem

`/proc` is not real files on disk — it’s a live, kernel-generated view into system and per-process state, and it’s what tools like `ps`, `top`, and `free` read under the hood.

```
cat /proc/loadavg                        # raw load average data
cat /proc/cpuinfo                        # detailed per-core CPU info
cat /proc/<pid>/status                    # detailed process state: memory, threads, signal masks
cat /proc/<pid>/limits                   # this process's effective ulimits
ls -l /proc/<pid>/fd/                    # every open file descriptor of a process – invaluable for "too many open files" debugging
cat /proc/<pid>/cmdline | tr '\0' ' '    # the exact command line a process was launched with
cat /proc/sys/net/ipv4/ip_forward        # kernel network tunables live under /proc/sys – this one controls IP forwarding
cat /proc/<pid>/cgroup                    # which cgroups (hence which Docker container / K8s pod) this process belongs to
```

```
# "Too many open files" triage
cat /proc/<pid>/limits | grep "open files"      # what's the configured limit?
ls /proc/<pid>/fd | wc -l
# how many are actually open right now?
ulimit -n                                       # current shell's soft limit
```

9.5 Disk I/O Performance

```
iostat -xz 1 5                                # extended stats, 5 samples: %util (how busy the
device is), await (latency), r/s, w/s (needs sysstat)
iotop                                          # like top, but for disk I/O per-process (needs
root, install iotop package)
sar -d 1 5                                    # historical/live disk activity (also sysstat)
- sar can also report from logged history: sar -d -f /var/log/sysstat/saXX
```

iostat column	Meaning	What a bad value tells you
<code>%util</code>	Percentage of time the device was busy servicing I/O	Near 100% sustained = disk is the bottleneck
<code>await</code>	Average I/O request latency (ms), including queue time	High await + normal %util can indicate a slow/throttled cloud volume
<code>r/s</code> , <code>w/s</code>	Reads/writes per second	Compare against your cloud volume's provisioned IOPS limit

PRODUCTION SCENARIO: THROTTLED CLOUD VOLUME

On AWS, a gp3 EBS volume has a provisioned IOPS/throughput ceiling. When an app hits it, you will see `%util` near 100% and rapidly climbing `await` in `iostat`, while the application simply appears “slow” with no errors. The fix is at the infrastructure layer (increase provisioned IOPS/throughput, or move to a faster volume type) — no amount of application tuning fixes a throttled disk.

9.6 Cheat Sheet: “Something’s Wrong” First Five Commands

Symptom	First commands to run
Host generally slow	<code>uptime</code> then <code>top</code> / <code>htop</code> then <code>vmstat 1 5</code>
Suspected memory pressure	<code>free -h</code> then <code>dmesg -T \ grep -i oom</code> then <code>ps --sort=-rss</code>
Suspected disk I/O bottleneck	<code>iostat -xz 1 5</code> then <code>iotop</code>
Disk full	<code>df -h</code> then <code>df -ih</code> (inodes!) then <code>du -sh /*</code> <code>2>/dev/null \ sort -h</code>
“Can’t connect” to a service	<code>ss -tulpn \ grep <port></code> then <code>curl -v</code> <code>localhost:<port></code> then <code>journalctl -u</code> <code><service></code>
Service won’t start	<code>systemctl status <service></code> then <code>journalctl -u <service> -n 100</code>

10. Log Management

10.1 What It Is and Why It Exists

Logs are the primary record of what a system did, and reliable log management (collection, rotation, retention, querying) is what turns “we think it crashed around 3am” into “here is the exact error, stack trace, and the request that triggered it.” Modern systemd-based distros centralize logs in the **journal**; older-style logging still uses **rsyslog** writing to plain text files under `/var/log`; and **logrotate** prevents any of it from filling the disk.

10.2 journalctl: Querying the systemd Journal

```
journalctl -u nginx.service          # all logs for one unit
journalctl -u nginx.service -f       # follow live (like tail -f)
journalctl -u nginx.service --since "1 hour ago" --until "30 min ago"
journalctl -u nginx.service -p err   # only priority "err" and above
(emerg,alert,crit,err)
journalctl -k                        # kernel messages only (dmesg
equivalent, persisted)
journalctl -b                        # logs since the current boot
only
journalctl -b -1                     # logs from the PREVIOUS
boot – essential after an unexpected reboot
journalctl --disk-usage               # how much space the
journal itself is consuming
journalctl -o json-pretty -u nginx.service -n 5
# structured JSON output – great for piping into jq or a log shipper
```

```
# /etc/systemd/journald.conf – control journal size and persistence
[Journal]
Storage=persistent      # default on many distros is volatile (RAM-only, lost on
reboot) – persistent survives reboots
SystemMaxUse=2G         # hard cap on total disk space the journal may use
MaxRetentionSec=2week   # auto-vacuum entries older than this
```

10.3 rsyslog and /var/log

Many services (and all of `/var/log/auth.log`, `/var/log/syslog` / `/var/log/messages`) still flow through **rsyslog**, which routes messages based on **facility** (subsystem: `auth`, `cron`, `kern`, `daemon` ...) and **severity** (`emerg` down to `debug`) to files, remote servers, or the journal.

```
tail -f /var/log/syslog          # Debian/Ubuntu general system log
tail -f /var/log/messages       # RHEL/CentOS equivalent
tail -f /var/log/auth.log
# authentication events (SSH logins, sudo usage) – Debian/Ubuntu
tail -f /var/log/secure         # RHEL/CentOS equivalent of auth.log

grep "Failed password" /var/log/auth.log | tail -20  # quick brute-force / failed-
login check
```

```
# /etc/rsyslog.d/50-custom.conf – route a specific facility to its own file, and also
forward to a central log server
local0.*    /var/log/myapp.log
*.*         @@logs.internal.example.com:514      # @@ = TCP forwarding to a remote
syslog collector
```

10.4 logrotate: Preventing Disk-Filling Logs

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14          # keep 14 rotated files
    compress
    delaycompress      # don't compress the most-recently-rotated file (still
might be read by a slow tail -F)
    missingok
    notifempty
    create 0640 appsvc appsvc
    sharedscripts
    postrotate
        systemctl reload myapp > /dev/null 2>&1 || true    # tell the app to reopen its
log file handle
    endscript
}
```

```
sudo logrotate -d /etc/logrotate.d/myapp    # DRY RUN – shows what would happen
without doing it
sudo logrotate -f /etc/logrotate.d/myapp    # force rotation now, useful to test
the postrotate hook
cat /var/lib/logrotate/status               # when each config last ran
```

COMMON MISTAKE: ROTATION WITHOUT REOPENING THE FILE HANDLE

If `postrotate` doesn't signal the application to reopen its log file (or you don't use `copytruncate`), the app keeps writing into the now-renamed/deleted inode forever — `myapp.log` stays at 0 bytes while disk usage silently climbs (this connects directly back to the inode discussion in Section 3.3). Either send the app a reload signal in `postrotate`, or use the `copytruncate` directive if the app can't be signaled (with the small tradeoff of a possible few lost log lines during the truncate).

11. SSH & Remote Access

11.1 What It Is and Why It Exists

SSH (Secure Shell) is the encrypted channel through which essentially all Linux server administration happens — interactive shells, file transfer, port forwarding, and (critically) the transport for Git and most CI/CD deploy mechanisms. Key-based authentication, proper `sshd_config` hardening, and comfort with tunneling are non-negotiable DevOps skills.

11.2 Key-Based Authentication

```
ssh-keygen -t ed25519 -C "dhanush@laptop"           # generate a modern, fast, secure
keypair (prefer over RSA today)
# -> ~/.ssh/id_ed25519 (private, NEVER share) and ~/.ssh/id_ed25519.pub (public, safe
to share)

ssh-copy-id -i ~/.ssh/id_ed25519.pub deploy@10.0.1.20
# install your public key into the remote's authorized_keys

cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys   # manual equivalent, on the
remote host
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys   # SSH refuses to use
keys/authorized_keys with overly loose permissions

ssh-agent bash                                       # start an agent to
hold decrypted keys in memory for the session
ssh-add ~/.ssh/id_ed25519                           # load a key into
the running agent (prompts for passphrase once)
ssh-add -l                                           # list keys
currently loaded in the agent
```

11.3 The SSH Client Config

```
# ~/.ssh/config – define per-host shortcuts, jump hosts, and defaults
Host prod-bastion
    HostName bastion.example.com
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    Port 22

Host prod-app-*
    ProxyJump prod-bastion          # transparently hop through the bastion to reach
private hosts                      # private hosts
    User deploy
    IdentityFile ~/.ssh/id_ed25519

Host *
    ServerAliveInterval 30          # send a keepalive every 30s (prevents idle
NAT/firewall timeouts)            # NAT/firewall timeouts)
    ServerAliveCountMax 3
```

```
ssh prod-app-01                    # now just works – no need to remember IPs, users,
or the bastion hop
```

11.4 Hardening sshd

```
# /etc/ssh/sshd_config – key production hardening settings
PermitRootLogin no                  # never allow direct root SSH login
PasswordAuthentication no          # keys only – disables the entire brute-force-
able password attack surface
PubkeyAuthentication yes
AllowUsers deploy jenkins          # explicit allow-list of users who may SSH
in at all
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
```

```
sudo sshd -t                      # validate config syntax BEFORE reloading – a
typo here can lock out all SSH access
sudo systemctl reload sshd
```

CRITICAL SAFETY PRACTICE

Before reloading `sshd` with a hardened config, always keep your CURRENT session open and test the new config from a SECOND, separate connection. If the second connection fails, you still have the first session to roll back the change. This single habit has saved countless engineers from a full out-of-band console recovery.

11.5 Tunneling and File Transfer

```
# Local port forward: reach a remote-only service (e.g., a DB only reachable from
inside the VPC) via localhost
ssh -L 5432:db-internal.local:5432 prod-bastion
# now connect to 127.0.0.1:5432 on YOUR machine – traffic is tunneled through the
bastion

# Remote port forward: expose a port on YOUR machine to the remote side (e.g., share a
local dev server)
ssh -R 8080:localhost:3000 prod-bastion

# Dynamic (SOCKS proxy) forward – route arbitrary traffic through the tunnel
ssh -D 1080 prod-bastion

scp file.tar.gz deploy@prod-app-01:/opt/releases/          # simple one-off copy
scp -r ./dist/ deploy@prod-app-01:/opt/app/                # recursive directory copy

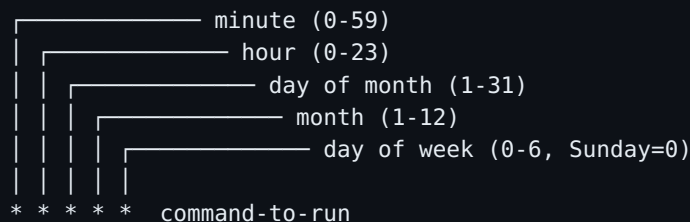
rsync -avz --progress ./dist/ deploy@prod-app-01:/opt/app/
# preferred over scp for anything repeated: only transfers diffs
rsync -avz --delete ./dist/ deploy@prod-app-01:/opt/app/  # --delete: mirror
exactly, removing files gone from source (use carefully)
```

12. Scheduling: cron, at, and systemd Timers

12.1 What It Is and Why It Exists

Operational automation — backups, cleanup jobs, health audits, certificate renewal — needs to run on a schedule without a human triggering it. **cron** is the traditional, universally available scheduler; **at** runs a one-off job in the future; **systemd timers** (Section 5.4.3) are the modern systemd-native alternative with better logging and dependency handling.

12.2 crontab Syntax



A diagram illustrating the crontab syntax. It shows a sequence of five asterisks (*) representing the fields: minute, hour, day of month, month, and day of week. Each field is connected by a vertical line to a label and its range: minute (0-59), hour (0-23), day of month (1-31), month (1-12), and day of week (0-6, Sunday=0). Below these fields, the text 'command-to-run' is shown, indicating that the asterisks are placeholders for the actual command to be executed.

```
crontab -e          # edit the CURRENT user's crontab
crontab -l          # list current user's cron jobs
crontab -u deploy -l # list another user's crontab (needs root)
sudo crontab -e     # edit root's crontab

# System-wide cron locations
/etc/crontab        # system crontab — has an EXTRA "user" field before
the command
/etc/cron.d/*       # drop-in system cron files (preferred for
packaged/automated jobs)
/etc/cron.daily/, /etc/cron.hourly/
# scripts here run automatically at that cadence (no crontab syntax needed)
```

```
# Examples
0 2 * * *          /opt/scripts/backup.sh >> /var/log/backup.log 2>&1    # every day at
02:00
*/15 * * * *       /opt/scripts/health_check.sh                        # every 15
minutes
0 9 * * 1-5        /opt/scripts/weekday_report.sh                      # 9 AM,
Monday-Friday only
0 0 1 * *          /opt/scripts/monthly_cleanup.sh                      #
midnight on the 1st of every month
```

COMMON MISTAKE #1: MISSING ENVIRONMENT

cron jobs run with a minimal environment — no `PATH` beyond `/usr/bin:/bin`, no shell profile sourced, no interactive-shell variables. A script that works fine when you run it manually can fail silently under cron because it can't find a binary or a variable it expects. Always use absolute paths in scripts, and explicitly set any required environment variables at the top of the crontab or inside the script itself.

COMMON MISTAKE #2: SILENT FAILURES

By default, cron emails job output to the user's local mailbox — which on most modern servers nobody ever reads, so failures go completely unnoticed. Always redirect output explicitly (`>> /var/log/job.log 2>&1`) and add real alerting: have the script itself call a monitoring/heartbeat endpoint (e.g., a “dead man's switch” style check like Healthchecks.io, or push a metric/alert to your monitoring stack) on both success and failure.

```
# A defensively-written cron entry
5 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1 || curl -fsS -X POST
https://alerts.internal/webhook -d "backup.sh failed on $(hostname)"
```

12.3 at: One-Off Future Jobs

```
echo "systemctl restart myapp" | at 23:30          # run once, tonight at 23:30
at now + 1 hour <<< "/opt/scripts/cleanup_tmp.sh"  # run once, one hour from
now
atq                                              # list pending at jobs
atrm 3
# cancel a pending job by its queue ID
```

12.4 cron vs systemd Timers vs Kubernetes CronJobs

	cron	systemd timer	Kubernetes CronJob
Scope	Single host	Single host	Cluster-wide, containerized
Logging	Manual (own redirection)	Integrated into <code>journalctl</code>	Pod logs, <code>kubectl logs</code> , standard K8s observability
Dependency ordering	None	Full systemd <code>After=</code> / <code>Requires=</code>	K8s scheduling + init containers
Missed-run recovery	No (job simply doesn't run if host was off)	Yes, via <code>Persistent=true</code>	Yes, via <code>startingDeadlineSeconds</code> + <code>concurrencyPolicy</code>
Best for	Simple, single-host legacy scripts	Any modern single-host scheduled task tied to a systemd service	Any scheduled task in a containerized/K8s environment (the default modern choice)

Part 09 of this series covers Kubernetes CronJobs in full depth, including concurrency policy and history limits. # 13. Bash Scripting Fundamentals

13.1 What It Is and Why It Exists

Bash scripting is how you turn a sequence of manual commands into something repeatable, reviewable, and automatable — the glue between every tool in this series. CI/CD pipelines, Ansible `shell` tasks, Docker `ENTRYPOINT` scripts, and Kubernetes init containers all ultimately run Bash (or a Bash-like shell) under the hood. Weak Bash skills mean brittle automation; strong ones mean you can write scripts that fail loudly and safely instead of silently corrupting state.

13.2 The Shebang and Execution

```
#!/usr/bin/env bash
# The shebang line – tells the kernel which interpreter to run this file with.
# Prefer "#!/usr/bin/env bash" over a hardcoded "#!/bin/bash" – it finds bash via
$PATH,
# which matters on systems where bash isn't at exactly /bin/bash (rare on Linux, common
on some minimal images).

chmod +x script.sh      # make it executable
./script.sh             # run it (needs the shebang + execute bit)
bash script.sh          # run it explicitly with bash, ignoring the shebang and
execute bit
```

13.3 Variables, Quoting, and Substitution

```
name="Dhanush"           # NO spaces around = – this is the #1 syntax error
beginners hit
echo "Hello, $name"       # "Hello, Dhanush"
echo "Hello, ${name}!"    # braces avoid ambiguity when concatenating: $
{name}suffix

readonly MAX_RETRIES=5    # constant – reassignment will error
export DEPLOY_ENV=production # exported – visible to child processes/
subshells, not just this shell

# Command substitution – capture a command's output into a variable
current_date=$(date +%Y-%m-%d)
files_count=$(ls /var/log | wc -l)

# Arithmetic
count=$((count + 1))
result=$(( 10 / 3 ))      # integer division only – result is 3
```

Quoting rules (the single most common source of Bash bugs)

Quote style	Behavior
"double quotes"	Variables and command substitution ARE expanded; use for almost everything
'single quotes'	Nothing is expanded — completely literal; use for regexes, literal strings
No quotes	Variables expand AND undergo word-splitting + glob expansion — dangerous with filenames/paths containing spaces

```
file="my backup.tar.gz"
rm $file          # BROKEN: expands to  rm my backup.tar.gz  → tries to delete TWO
files: "my" and "backup.tar.gz"
rm "$file"        # CORRECT: expands to  rm "my backup.tar.gz" → deletes the one
intended file

for f in $(ls *.txt); do ...
# BROKEN pattern – breaks on filenames with spaces, and is an unnecessary ls call
for f in *.txt; do ... done          # CORRECT – direct globbing, handles spaces safely
```

RULE OF THUMB

Always double-quote variable expansions (`"$var"`, `"$@"`) unless you specifically want word-splitting/globbing. This one habit eliminates a large fraction of real-world Bash bugs, especially around filenames and paths.

13.4 Conditionals

```
if [ "$env" == "production" ]; then
    echo "Deploying to prod"
elif [ "$env" == "staging" ]; then
    echo "Deploying to staging"
else
    echo "Unknown environment: $env"
    exit 1
fi

# [[ ]] is the modern Bash-specific test – prefer it over [ ]: safer word-splitting,
supports && || directly, regex match
if [[ "$env" == "production" && "$force" == "true" ]]; then
    echo "Forced prod deploy"
fi

if [[ "$version" =~ ^v[0-9]+\.[0-9]+\.[0-9]+$ ]]; then      # regex match
    echo "Valid semver: $version"
fi
```

Test	Meaning
<code>-f "\$file"</code>	Regular file exists
<code>-d "\$dir"</code>	Directory exists
<code>-e "\$path"</code>	Path exists (any type)
<code>-x "\$file"</code>	File exists and is executable
<code>-z "\$str"</code>	String is empty
<code>-n "\$str"</code>	String is non-empty
<code>-eq -ne -lt -le -gt -ge</code>	Numeric comparisons
<code>== !=</code>	String comparisons (inside <code>[[]]</code>)

```
[[ -f "$config" ]] || { echo "Config missing: $config" >&2; exit 1; }
# a common one-liner idiom: run the right side ONLY if the left side fails
```

13.5 Loops

```
for i in {1..5}; do echo "Attempt $i"; done

for server in web01 web02 web03; do
    ssh "$server" "systemctl restart myapp"
done

# C-style loop
for ((i=0; i<5; i++)); do echo "$i"; done

# while loop – common for retry logic and reading files
attempt=1
while [[ $attempt -le 5 ]]; do
    if curl -sf http://localhost:8080/health; then
        echo "Healthy"
        break
    fi
    echo "Attempt $attempt failed, retrying..."
    ((attempt++))
    sleep 2
done

# Read a file line by line – THE correct, safe pattern
while IFS= read -r line; do
    echo "Processing: $line"
done < input.txt
# IFS= preserves leading/trailing whitespace; -r prevents backslash interpretation
```

13.6 Functions

```
log_info() {
    local msg="$1"
    # ALWAYS use `local` inside functions –
    # otherwise variables leak into global scope
    echo "[INFO] $(date '+%Y-%m-%d %H:%M:%S') - $msg"
}

deploy_service() {
    local service_name="$1"
    local version="$2"

    if [[ -z "$service_name" || -z "$version" ]]; then
        echo "Usage: deploy_service <name> <version>" >&2
        return 1
        # `return` inside a function, NOT `exit`
        # (exit would kill the whole script)
    fi

    log_info "Deploying $service_name version $version"
    # ... actual deploy logic ...
    return 0
}

deploy_service "api" "1.4.2"
echo "Exit code: $?" # $? = the exit status of the LAST command run
```

13.7 Positional Arguments and getopt

```
#!/usr/bin/env bash
echo "Script name: $0"
echo "First arg: $1"
echo "All args as separate words: $@"
echo "Number of args: $#"
```



```
# getopt – proper flag parsing (-e, -v, --help style single-letter flags)
usage() { echo "Usage: $0 -e <environment> [-v] [-h]"; exit 1; }
```



```
verbose=false
while getopt "e:vh" opt; do
    case "$opt" in
        e) environment="$OPTARG" ;;
        v) verbose=true ;;
        h) usage ;;
        *) usage ;;
    esac
done
```



```
[[ -z "$environment" ]] && usage
echo "Deploying to: $environment (verbose=$verbose)"
```

```
./deploy.sh -e production -v
```

13.8 Exit Codes

```
exit 0      # success – the ONLY code that means "everything worked" by convention
exit 1      # general error (the most common non-zero code)
exit 2      # often used for misuse of shell builtins / bad usage
# 126 = command found but not executable
# 127 = command not found
# 128+N = terminated by signal N (e.g., 137 = 128+9 = killed by SIGKILL, 143 = 128+15 = SIGTERM)

command1 && command2    # run command2 ONLY IF command1 succeeded (exit code 0)
command1 || command2    # run command2 ONLY IF command1 FAILED (exit code != 0)
command1; command2      # run both, regardless of command1's result

echo $?                # always check this immediately after a command whose
                        # success you care about
```

WHY THIS MATTERS EVERYWHERE

Exit codes are the contract every automation layer relies on: a CI/CD pipeline step is marked failed purely because a script exited non-zero; a systemd service with `Restart=on-failure` only restarts on non-zero exit; a Kubernetes container's `livenessProbe` exec check considers non-zero unhealthy. A script that always exits 0 "to keep the pipeline green" is one of the most damaging automation anti-patterns — it hides real failures from every layer built on top of it.

13.9 set -euo pipefail: The Production Standard Header

```
#!/usr/bin/env bash
set -euo pipefail
# -e : exit immediately if any command fails (non-zero exit) – no silent continuation
# past an error
# -u : treat unset variables as an error instead of silently expanding to an empty
# string (catches typos in var names)
# -o pipefail : a pipeline's exit status is the LAST non-zero command in it, not just
# the final command
# (without this, "false | true" exits 0 – the failure of `false` is
# hidden)

IFS=$'\n\t'      # optional but common companion: restrict word-splitting to
                 # newlines/tabs, not spaces
```

CAVEATS OF SET -E

`set -e` does NOT trigger inside a condition already being checked (`if mycommand; then`), inside `&&` / `||` chains, or inside a function called as part of one of those — this trips people up constantly. Also, a command whose failure you're deliberately ignoring must be marked explicitly: `rm -f maybe_missing_file.txt || true`. Always start a production script with `set -euo pipefail` as a default, but understand these exceptions rather than assuming it catches every possible failure.

13.10 Traps: Cleanup and Signal Handling

```
#!/usr/bin/env bash
set -euo pipefail

TMP_DIR=$(mktemp -d)

cleanup() {
    echo "Cleaning up temp directory: $TMP_DIR"
    rm -rf "$TMP_DIR"
}

trap cleanup EXIT
# runs on ANY exit: normal completion, error (set -e), or signal – guarantees cleanup

trap 'echo "Interrupted, aborting."; exit 130' INT TERM      # custom handling for
Ctrl+C / SIGTERM

# ... script body using $TMP_DIR ...
echo "Working in $TMP_DIR"
```

WHY THIS MATTERS

`trap ... EXIT` is the Bash equivalent of a `finally` block — it is the correct, reliable way to guarantee temp files are removed, lock files released, or a maintenance-mode flag is cleared, no matter how the script terminates. Scripts that only clean up at the “end” of the file (and not in a trap) leave garbage behind on every error path.

14. Text Processing & Real Automation Scripts

14.1 What It Is and Why It Exists

Most operational Bash scripting is really just text processing: parsing logs, extracting fields, filtering output, transforming config files. `grep`, `sed`, `awk`, `find`, `xargs`, and their friends are the actual daily-driver toolkit — far more than loops and functions in isolation. Fluency here is what makes a one-liner solve in two minutes what would otherwise be a 40-line script.

14.2 `grep`: Finding Lines

```
grep "ERROR" app.log           # lines containing "ERROR"
grep -i "error" app.log        # case-insensitive
grep -v "DEBUG" app.log        # INVERT – lines NOT containing
                                "DEBUG"
grep -c "ERROR" app.log        # COUNT matching lines instead of
                                printing them
grep -n "ERROR" app.log        # show line numbers
grep -r "TODO" ./src/          # recursive directory search
grep -E "ERROR|WARN|FATAL" app.log # extended regex – match any
                                of several patterns
grep -B2 -A5 "OutOfMemory" app.log
# show 2 lines BEFORE and 5 lines AFTER each match – essential for stack traces
grep -oP '(?<=user_id=)\d+' app.log # -o: only the matched
                                text, -P: Perl regex with a lookbehind – extract just the value
grep --include="*.yaml" -r "image:" ./k8s/ # limit recursive
                                search to specific file types
```

14.3 sed: Stream Editing

```
sed 's/foo/bar/' file.txt          # replace FIRST occurrence of "foo" with
"bar" on each line
sed 's/foo/bar/g' file.txt
# replace ALL occurrences (g = global) on each line
sed -i 's/foo/bar/g' file.txt      # -i = edit the file IN PLACE (no
output to stdout)
sed -i.bak 's/foo/bar/g' file.txt
# in-place, but keep a .bak backup first – safer for production edits

sed -n '10,20p' file.txt          # print only lines 10 through 20
sed '/^#/d' config.conf           # delete lines starting with #
(comment)
sed -i '/^$/d' file.txt           # delete all blank lines in place

# Real example: bump a version string across a config file
sed -i "s/image: myapp:./image: myapp:${NEW_VERSION}/" deployment.yaml
```

14.4 awk: Field-Based Processing

awk treats each line as a record split into fields (**\$1**, **\$2**, ... **\$0** = whole line), which makes it the right tool for anything column-oriented — log parsing, **ps** / **df** output, CSV-like data.

```
awk '{print $1, $3}' access.log    # print columns 1 and 3 (default
delimiter: whitespace)
awk -F',' '{print $2}' data.csv    # use comma as the field
delimiter

awk '{print $NF}' file.txt         # NF = number of fields; $NF
= the LAST field on each line
awk '{sum += $5} END {print sum}' access.log # sum column 5 across all
lines, print the total at the end

awk '$9 == 500 {print $1, $7}' access.log # filter: only lines
where field 9 (HTTP status) equals 500
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head # top talkers:
extract IP, count occurrences, sort descending

# Real example: total memory used by a specific process name from ps output
ps aux | awk '/nginx/ && !/awk/ {sum += $6} END {print sum/1024 " MB"}'
```

14.5 find and xargs

```
find /var/log -name "*.log" -mtime +30                # files matching *.log, last
modified more than 30 days ago
find /var/log -name "*.log" -size +100M              # files larger than 100MB
find /tmp -type f -mtime +7 -delete
# delete files older than 7 days (careful – test with -print first!)
find / -xdev -perm -4000 -type f 2>/dev/null          # SUID binaries,
staying on one filesystem (-xdev), errors suppressed

find /var/log -name "*.log.gz" -mtime +30 -print      # ALWAYS dry-run with
-print before adding -delete or piping to rm
find /var/log -name "*.log.gz" -mtime +30 -exec rm {} \; # -exec: run a
command per result (slower, one process per file)
find /var/log -name "*.log.gz" -mtime +30 | xargs rm -v # xargs: batches
arguments into fewer process invocations (faster)
find . -name "*.tmp" -print0 | xargs -0 rm -f          # -print0 /
xargs -0: NUL-delimited – the safe way to handle filenames with spaces/newlines
```

SAFETY HABIT

Any `find ... -delete` or `find ... | xargs rm` command should be run first WITHOUT the destructive part (just `-print`, or pipe to `xargs echo`) to see exactly what would be affected. This costs ten seconds and has prevented countless “I deleted the wrong files in production” incidents.

14.6 Other Everyday Tools

```
cut -d',' -f1,3 data.csv                # extract fields 1 and 3, comma-delimited
sort -k2 -n file.txt                    # sort by the 2nd column, numerically
sort -u file.txt                        # sort and remove duplicates
uniq -c sorted_file.txt                 # count consecutive duplicate lines
(MUST be sorted first)
tr 'a-z' 'A-Z' < file.txt               # translate lowercase to uppercase
wc -l file.txt                          # count lines
tee output.log
# write to a file AND pass through to stdout simultaneously – useful mid-pipeline
column -t -s',' data.csv                # pretty-align delimited data
into columns
jq '.items[].name' response.json
# the standard tool for JSON – essential for API/kubectl -o json output
watch -n 2 'kubectl get pods'           # re-run a command every 2
seconds, full-screen – live monitoring without a loop
```

14.7 Complete Working Script #1: Server Health Check

```
#!/usr/bin/env bash
# health_check.sh – reports CPU, memory, disk, and service status; alerts on threshold
breach
set -euo pipefail

readonly DISK_THRESHOLD=85
readonly MEM_THRESHOLD=90
readonly SERVICES=("nginx" "myapp" "docker")
readonly LOG_FILE="/var/log/health_check.log"
readonly ALERT_WEBHOOK="${ALERT_WEBHOOK:-}" # ${VAR:-default} – use $VAR if
set, else empty string

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOG_FILE"; }

alert() {
    local message="$1"
    log "ALERT: $message"
    if [[ -n "$ALERT_WEBHOOK" ]]; then
        curl -fsS -X POST "$ALERT_WEBHOOK" -H "Content-Type: application/json" \
            -d "{\"text\": \"[$(hostname)] $message\"}" || log "Failed to send alert
webhook"
    fi
}

check_disk() {
    while read -r usage mount; do
        usage="${usage%\\%}" # strip the trailing %
        character
        if (( usage >= DISK_THRESHOLD )); then
            alert "Disk usage on $mount is ${usage}% (threshold: ${DISK_THRESHOLD}%)"
        fi
    done < <(df -h --output=pcent,target | tail -n +2)
}

check_memory() {
    local mem_used_pct
    mem_used_pct=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
    if (( mem_used_pct >= MEM_THRESHOLD )); then
        alert "Memory usage is ${mem_used_pct}% (threshold: ${MEM_THRESHOLD}%)"
    fi
}

check_services() {
    for svc in "${SERVICES[@]}; do
        if ! systemctl is-active --quiet "$svc"; then
            alert "Service $svc is NOT running"
        fi
    done
}

main() {
    log "Starting health check on $(hostname)"
    check_disk
    check_memory
    check_services
    log "Health check complete"
}
```

```
}  
  
main "$@"
```

```
# Deploy as a systemd timer (preferred) – see Section 5.4.3 for the  
paired .service/.timer files  
# Or as cron: */5 * * * * ALERT_WEBHOOK=https://hooks.example.com/x /opt/scripts/  
health_check.sh
```

14.8 Complete Working Script #2: Docker Image & Log Cleanup

```
#!/usr/bin/env bash
# cleanup.sh – reclaims disk space from old Docker images and rotated logs on a VM
set -euo pipefail

readonly LOG_RETENTION_DAYS=14
readonly LOG_DIR="/var/log/myapp"

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"; }

cleanup_docker() {
    log "Removing dangling images..."
    docker image prune -f

    log "Removing images unused for 72h+..."
    docker image prune -a -f --filter "until=72h"

    log "Removing stopped containers older than 24h..."
    docker container prune -f --filter "until=24h"

    log "Removing unused volumes (CAUTION: only run if you're sure nothing needs them)..."
    docker volume prune -f

    log "Removing unused networks..."
    docker network prune -f
}

cleanup_old_logs() {
    log "Removing logs older than ${LOG_RETENTION_DAYS} days from ${LOG_DIR}..."
    find "$LOG_DIR" -name "*.log.gz" -mtime "+${LOG_RETENTION_DAYS}" -print -delete
}

report_disk_usage() {
    log "Disk usage after cleanup:"
    df -h / /var
    log "Docker disk usage:"
    docker system df
}

main() {
    log "Starting cleanup on $(hostname)"
    cleanup_docker
    cleanup_old_logs
    report_disk_usage
    log "Cleanup complete"
}

main "$@"
```

PRODUCTION NOTE

This script's Docker-specific portion is the exact pattern referenced later in this series under "Docker image cleanup on VMs." Run it via a systemd timer weekly, and always alert-on-failure (wrap the cron/timer invocation the same way as the health check script) rather than assuming a cleanup job silently succeeded.

15. Security Hardening Basics

15.1 What It Is and Why It Exists

A DevOps engineer is frequently the last line of defense on host-level security — cloud security groups get the headlines, but a poorly hardened OS is still the easiest way into a system. This section covers the host-level controls every production Linux server should have, independent of any cloud-specific controls (covered in later parts).

15.2 Baseline Hardening Checklist

```
# 1. Keep packages patched – automate this, don't rely on memory
sudo apt update && sudo apt upgrade -y          # Debian/Ubuntu
sudo dnf upgrade -y                             # RHEL/Amazon Linux

# Unattended security upgrades (Debian/Ubuntu)
sudo apt install unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades

# 2. Disable root SSH login and password auth (Section 11.4)
# 3. Restrict sudo to specific commands, not blanket access (Section 4.4)
# 4. Enforce a firewall default-deny posture (Section 8.7)
# 5. Remove unused packages/services – smaller attack surface
systemctl list-unit-files --state=enabled | grep -v "^UNIT"    # review everything
set to start at boot
sudo apt autoremove --purge                                   # remove unused
packages entirely
```

15.3 fail2ban: Automated Brute-Force Protection

fail2ban watches log files (SSH auth logs, web server logs) for repeated failure patterns and temporarily bans the offending IP at the firewall level.

```
sudo apt install fail2ban

# /etc/fail2ban/jail.local (never edit jail.conf directly – it gets overwritten on
upgrade)
[sshd]
enabled = true
port = 22
maxretry = 4
findtime = 600          # within this many seconds...
bantime = 3600          # ...ban for this long

sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd          # see currently banned IPs and stats
sudo fail2ban-client set sshd unbanip 1.2.3.4  # manually unban
```

15.4 SELinux and AppArmor: Mandatory Access Control

Standard Unix permissions (Section 4) are **discretionary** — the owner decides access. **Mandatory Access Control (MAC)** systems add a kernel-enforced policy layer on top that even root cannot casually bypass, drastically limiting what a compromised process can actually do, even if it has “correct” Unix permissions.

	SELinux	AppArmor
Used by	RHEL/CentOS/Fedora/Amazon Linux	Ubuntu/Debian
Model	Label-based (every file/process gets a security context)	Path-based (profiles reference file paths directly)
Learning curve	Steeper	Gentler

```
# SELinux
getenforce          # Enforcing / Permissive / Disabled
sudo setenforce 0   # temporarily switch to Permissive (LOGS violations
but doesn't BLOCK – great for debugging)
ls -Z /var/www/html # view SELinux context/labels on files
sudo semanage port -a -t http_port_t -p tcp 8080 # allow a custom port for a
labeled service type
sudo ausearch -m avc -ts recent # find recent SELinux DENIALS – the actual
troubleshooting starting point
sudo sealert -a /var/log/audit/audit.log # human-readable explanation + suggested
fix for a denial (needs setroubleshoot)

# AppArmor
sudo aa-status # which profiles are loaded, and in enforce vs
complain mode
sudo aa-complain /etc/apparmor.d/usr.sbin.nginx # switch a profile to complain (log-
only) mode for debugging
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx # switch back to enforcing
```

COMMON MISTAKE

`setenforce 0` or disabling AppArmor entirely “to make the app work” is a workaround, not a fix — it silently removes a real security boundary. The correct process is: switch to permissive/complain mode, reproduce the issue, read the denial logs (`ausearch` / `aa-status`), generate a proper policy exception for exactly what’s needed, then re-enable enforcement. This exact SELinux-blocking-things pattern is one of the most common causes of “works on my machine, fails on the RHEL server” bugs.

15.5 Auditing and File Integrity

```
sudo auditctl -w /etc/passwd -p wa -k passwd_changes      # watch a file for write/
attribute changes, tagged with a searchable key
sudo ausearch -k passwd_changes                          # search audit log by
that key

# AIDE – file integrity monitoring (detects unexpected changes to system binaries/
configs)
sudo apt install aide
sudo aideinit                                             # build the initial baseline database
sudo aide --check
# compare current state against the baseline – flags tampering or drift
```

15.6 Security Cheat Sheet: Quick Host Audit

```
# Run this sequence on any host you're auditing for the first time
last -a | head -20                                       # recent login history
lastb | head -20                                         # recent FAILED login attempts
w                                                         # who's logged in right
now, and what they're doing
sudo cat /etc/passwd | awk -F: '$3 == 0 {print $1}'      # any non-root user with
UID 0?? (should be empty)
sudo find / -xdev -perm -4000 -type f 2>/dev/null        # SUID binaries
sudo find / -xdev -nouser -o -nogroup 2>/dev/null 2>&1   # files owned by a
since-deleted user/group
sudo ss -tulpn
# what's actually listening
sudo systemctl list-timers                               # what's
scheduled to run automatically
```

16. Troubleshooting Methodology

16.1 A Systematic Framework

Random command-firing under pressure is how incidents drag on. Use a consistent framework:

1. **Define the symptom precisely.** Not “the site is down” — “GET /checkout returns 502 for ~30% of requests, started at 14:12 UTC.”
2. **Establish scope.** One host or all? One service or everything? Since when exactly? What changed around that time (deploys, config changes, traffic pattern, infra changes)?
3. **Form a hypothesis, then check the cheapest thing that could confirm or rule it out.** Don’t restart things first “just in case” — you destroy the evidence you need to actually find the root cause.
4. **Work from the outside in, or the bottom up.** Either: user → LB → app → DB (follow the request), or: hardware/OS → network → process → application (follow the stack). Pick one and be systematic instead of jumping randomly between layers.
5. **Fix the immediate symptom (mitigate) separately from finding the root cause.** It’s fine to roll back a bad deploy to restore service, then investigate *why* it was bad, afterward, with logs/data you’ve preserved.
6. **Write it down.** A short incident timeline (what you observed, what you tried, what worked) turns into your postmortem and next time’s runbook.

16.2 Common Production Failure Scenarios

Scenario: Host load average is very high, but CPU usage looks normal

```
uptime                # load average high
vmstat 1 5             # check the 'b' column – processes blocked
(uninterruptible/D-state)
```

Diagnosis: load average counts D-state (disk-blocked) processes, not just runnable ones. High load + normal CPU% almost always means disk I/O contention. **Next step:**

`iostat -xz 1 5` → check `%util` and `await`. If a specific device is saturated, identify what’s writing to it (`iotop`), and check whether it’s a runaway log, a backup job overlapping with peak traffic, or a genuinely undersized/throttled volume.

Scenario: “Out of memory” but free -h shows memory available

```
free -h                # check "available" column specifically, not "free"
cat /proc/meminfo | grep -i commit    # CommitLimit vs Committed_AS – overcommit can
exhaust virtual memory limits before RAM
dmesg -T | grep -i "oom"              # confirm whether the OOM killer actually fired, and on
which process
```

Diagnosis: Either a specific cgroup/container hit ITS OWN memory limit (not host-wide) — check `docker stats` or `kubectl top pod` — or the host genuinely exhausted RAM+swap. `free -h`’s “available” figure and a per-cgroup check (`cat /sys/fs/cgroup/.../memory.current`) disambiguate host-level vs container-level OOM.

Scenario: Service is “running” (systemctl status = active) but not actually serving traffic

```
systemctl status myapp      # confirm PID exists, "active (running)"
ss -tulnp | grep <port>     # is it actually LISTENING on the expected port?
curl -v localhost:<port>/health # does it respond locally at all?
journalctl -u myapp -n 100 --no-pager # any errors right around startup or the
request?
```

Diagnosis: `active (running)` only means the main process hasn’t exited — it says nothing about application readiness. This gap is exactly why `Type=notify` (Section 5.4.2) and proper Kubernetes readiness probes (Part 13) exist — they check *actual* readiness, not just “process alive.”

Scenario: DNS resolution intermittently fails

```
dig example.com           # does it resolve at all right now?
for i in {1..10}; do dig +short example.com; sleep 1; done # is it flaky/
intermittent?
cat /etc/resolv.conf      # which resolvers are configured?
dig @<specific-resolver-ip> example.com
# test each configured resolver individually – is only ONE of them failing?
```

Diagnosis: Intermittent DNS failures are very often one resolver in a list of several being unhealthy, or a client-side connection-pooling/caching bug (e.g., glibc’s resolver not respecting TTLs correctly under certain configurations, or an application caching a DNS result far past its actual TTL, then a backing IP rotates — common with load balancers/RDS failover).

Scenario: Application crash-loops after a deploy

```
journalctl -u myapp -n 200 --no-pager      # get the actual error/stack trace from
the LAST failed start, not a stale one
systemctl status myapp                     # note the "Active" transition history
and restart count at the bottom
# If containerized:
docker logs --tail 200 <container>
kubectl logs <pod> --previous               # CRITICAL: --previous gets the
crashed container's logs, not the new restart's
```

Diagnosis: Almost always either (a) a bad config/env var from the new deploy, (b) a missing dependency/migration that should have run first, or (c) a resource limit that the new version's higher footprint now exceeds. Compare the failing version's diff against the last known-good deploy — this is why deploy metadata (what changed, when) matters as much as the logs themselves.

16.3 Systematic Debugging Tool Priority Order

Question	Tool
Is the process even running?	<code>ps</code> , <code>systemctl status</code>
Is it listening on the right port?	<code>ss -tulpn</code>
Can I reach it at all (any layer)?	<code>ping</code> → <code>nc -zv</code> → <code>curl -v</code>
What is it actually doing right now?	<code>strace -p</code> , <code>top</code> / <code>htop</code>
What did it log?	<code>journalctl -u</code> , <code>docker logs</code> , <code>kubectl logs</code>
What changed recently?	deploy history, <code>git log</code> , config management run history, <code>journalctl --since</code> around the incident time
Is it a resource constraint?	<code>free -h</code> , <code>iostat</code> , <code>vmstat</code> , <code>df -h</code>
Is it a permissions/security-policy block?	<code>dmesg</code> , <code>ausearch</code> (SELinux), <code>aa-status</code> (AppArmor), file <code>ls -l</code> ownership

THE META-LESSON

Every tool in this table maps directly onto a section earlier in this document. Troubleshooting isn't a separate skill from "knowing Linux" — it's the same knowledge applied under time pressure with a clear hypothesis-driven process instead of guesswork.

17. Hands-On Labs

LAB ENVIRONMENT

All labs assume a single Ubuntu 22.04+ or Amazon Linux 2023 VM (a free-tier cloud instance or a local VM/WSL is fine). Where a lab needs a second host, any second VM on the same network works.

Lab 1 — Boot, systemd, and a Custom Service

1. Write a tiny long-running script at `/opt/labapp/run.sh` that logs a timestamp to stdout every 5 seconds in a loop, and traps `SIGTERM` to log `"shutting down gracefully"` before exiting.
2. Create `/etc/systemd/system/labapp.service` with `Type=simple`, `Restart=on-failure`, running as a dedicated non-root system user you create with `useradd --system`.
3. `daemon-reload`, `enable --now`, confirm it's running with `systemctl status` and `journalctl -u labapp -f`.
4. Kill the process directly with `kill -9` and observe systemd auto-restart it. Check the restart count in `systemctl status`.
5. Set `systemctl set-property labapp.service MemoryMax=50M`, then write a small script that intentionally allocates memory past that limit, and observe the cgroup OOM kill in `dmesg -T` and `journalctl -u labapp`.

Lab 2 — Disk Full, Inode Exhaustion, and Recovery

1. Create a small (1GB) loopback-mounted filesystem: `dd if=/dev/zero of=/tmp/disk.img bs=1M count=1024`, `mkfs.ext4 /tmp/disk.img`, mount it at `/mnt/labdisk`.
2. Fill it to ~95% with a large file, confirm with `df -h /mnt/labdisk`.
3. Separately, on the same mount, write a script that creates 50,000 tiny empty files and observe `df -ih` inode usage climb, even with plenty of block space left (format with `-N` inode count to make this reproducible quickly, or just observe general inode usage).
4. Practice the “df says full, du says fine” scenario: start a background process that opens a file and writes to it continuously, `rm` the file while it's still open, confirm `df` still shows the space used, find the culprit with `lsdf +L1`, then kill the process and confirm space is reclaimed.

Lab 3 — LVM Live Resize

1. Attach two small extra virtual disks to your VM (or use loopback files as PVs for practice: `dd` two 2GB image files, `losetup` them as block devices).
2. Build the full stack: `pvcreate` → `vgcreate` → `lvcreate` (1.5GB) → `mkfs.xfs` → mount.

3. Fill it to ~90%, then live-extend: add the second PV to the VG, `lvextend`, `xfs_growfs` — all while a background `while true; do echo test >> /mnt/lvmtest/file.txt; sleep 1; done` loop keeps writing, to prove there's no downtime.

Lab 4 — Network Path Debugging

1. Stand up a simple web server on port 8080 on one VM (`python3 -m http.server 8080` is fine).
2. From a second VM, run the full diagnostic chain: `ping`, `nc -zv <host> 8080`, `curl -v http://<host>:8080`.
3. Now block port 8080 with the host firewall on the server VM (`ufw deny 8080` or an `iptables` DROP rule) and re-run the same three commands from the client — observe exactly which step starts failing and how the error differs (timeout vs refused vs no route).
4. Run `tcpdump -i any port 8080` on the server while retrying from the client, with the firewall rule in place, and observe whether the SYN packet even arrives — this proves whether the drop is happening at the firewall (packet never reaches the app) vs the application layer.

Lab 5 — Bash Automation: Log Analyzer

Write a script `log_report.sh` that, given an Nginx/Apache-style access log: 1. Reports total request count, and count broken down by HTTP status code family (2xx/3xx/4xx/5xx) using `awk`. 2. Reports the top 10 client IPs by request count. 3. Reports the top 10 slowest requests, if response time is a logged field. 4. Uses `set -euo pipefail`, validates the input file exists before processing, and exits with a non-zero code and a clear stderr message if it doesn't. 5. Accepts the log file path as a `getopts`-parsed `-f` flag, with a `-h` help option.

Lab 6 — Full Incident Simulation

1. Deploy the `labapp` service from Lab 1 again.
 2. Have a partner (or, solo, a second terminal acting “blind” later) introduce ONE of: (a) a bad `ExecStart` path, (b) a permissions issue on a file the service needs to read, (c) a port conflict with another process, (d) a memory limit set too low for the app to start.
 3. Using ONLY the troubleshooting framework in Section 16 (not trial-and-error), diagnose the root cause using `systemctl status`, `journalctl -u`, `ss -tulpn`, and file permission checks — and write a two-paragraph “incident summary” as if for a postmortem: symptom, root cause, fix, prevention.
-

18. Command & Reference Cheat Sheet

18.1 Process & Service Management

Command	Purpose
<code>ps aux</code> / <code>ps -ef</code>	List all processes
<code>top</code> / <code>htop</code>	Live process viewer
<code>systemctl status <svc></code>	Service state + recent logs
<code>systemctl enable --now <svc></code>	Enable at boot + start now
<code>journalctl -u <svc> -f</code>	Follow a unit's logs live
<code>kill -15 <pid></code> / <code>kill -9 <pid></code>	Graceful / forced termination
<code>systemctl set-property <svc> MemoryMax=512M</code>	Live cgroup resource limit

18.2 Disk & Filesystem

Command	Purpose
<code>df -hT</code>	Disk space by filesystem, human-readable
<code>df -ih</code>	Inode usage
<code>du -sh /* 2>/dev/null \ sort -h</code>	Find what's using disk space
<code>lsof +L1</code>	Find deleted-but-open files holding space
<code>lsblk -f</code>	Block devices + filesystem types
<code>mount -a</code>	Test/apply <code>/etc/fstab</code>

18.3 Networking

Command	Purpose
<code>ip a / ip r</code>	Interfaces / routing table
<code>ss -tulpn</code>	Listening ports + owning process
<code>dig +short <host></code>	Resolve a hostname
<code>curl -v <url></code>	Full request/response trace
<code>nc -zv <host> <port></code>	TCP port reachability check
<code>mtr <host></code>	Continuous traceroute + ping
<code>tcpdump -i eth0 -n port <port></code>	Packet capture

18.4 Bash Essentials

Pattern	Purpose
<code>set -euo pipefail</code>	Production script safety header
<code>"\$var"</code>	Always quote expansions
<code>trap cleanup EXIT</code>	Guaranteed cleanup on any exit path
<code>command1 \ exit 1</code>	Fail fast on error
<code>[[-f "\$file"]]</code>	File existence test
<code>while IFS= read -r line; do ... done < file</code>	Safe line-by-line file reading

18.5 Text Processing

Command	Purpose
<code>grep -B2 -A5 "pattern" file</code>	Context lines around a match
<code>sed -i.bak 's/old/new/g' file</code>	In-place replace, with backup
<code>awk -F',' '{print \$2}' file</code>	Field extraction
<code>find . -mtime +30 -print</code>	Find old files (dry run first)
<code>sort \ uniq -c \ sort -rn</code>	Frequency count, most common first

19. Production Readiness Checklist

Use this before considering any Linux host “production ready”:

- ☐ Root SSH login disabled; password auth disabled; key-based auth only
- ☐ `sudo` access scoped to specific commands per role, not blanket root
- ☐ Host firewall enabled with default-deny + explicit allow rules; SSH rule verified from a second session before enabling
- ☐ Automatic security patching enabled, or a documented manual patch cadence
- ☐ `fail2ban` (or equivalent) protecting SSH and any other exposed auth endpoints
- ☐ Journal/syslog set to `Storage=persistent` with a sane `SystemMaxUse`
- ☐ `logrotate` configured for every application log directory, with tested `postrotate` reload hooks
- ☐ Disk and inode usage monitored with alerting thresholds (not just disk — inodes too)
- ☐ Memory and OOM events monitored/alerted, not just discovered after the fact in `dmesg`
- ☐ Every critical service managed by systemd with an appropriate `Restart=` policy and correct `Type=`
- ☐ Cron/timer jobs redirect output to a log file AND alert on non-zero exit — no silent failures
- ☐ `/etc/fstab` entries for non-critical/network mounts include `nofail`
- ☐ SUID binary audit performed and baselined; unexpected SUID binaries investigated
- ☐ SELinux/AppArmor enabled and enforcing (not disabled “to make things work”)
- ☐ All production scripts use `set -euo pipefail`, quote variable expansions, and use `trap` for cleanup
- ☐ A documented, tested incident-response first-command sequence exists for the team (Section 16.3)